Application Study: Flash Attention

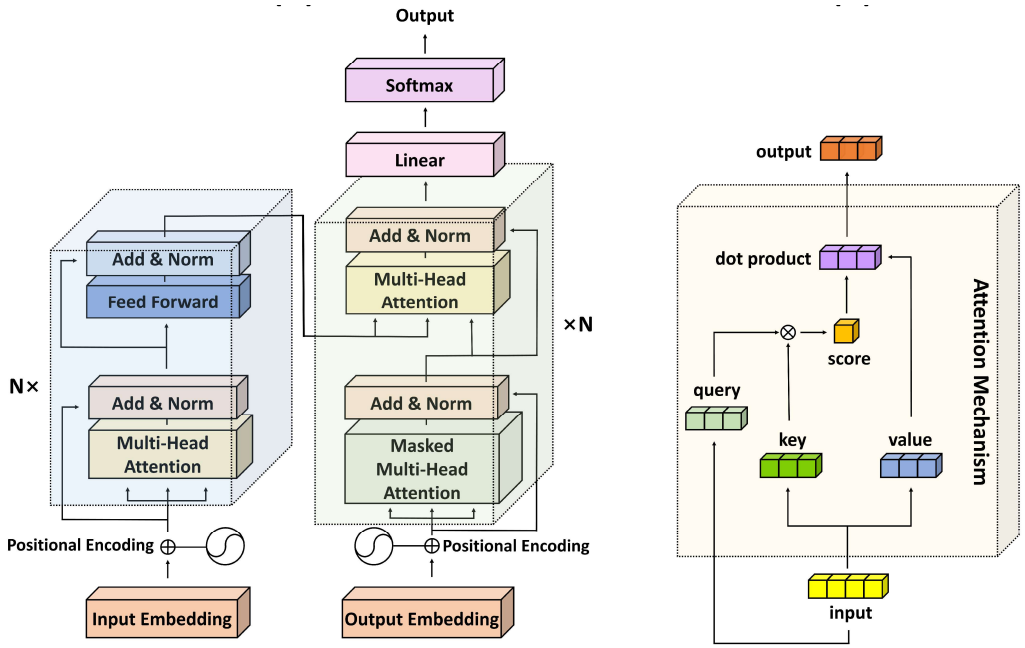
2026 spring

Jinho Lee

ECE, SNU

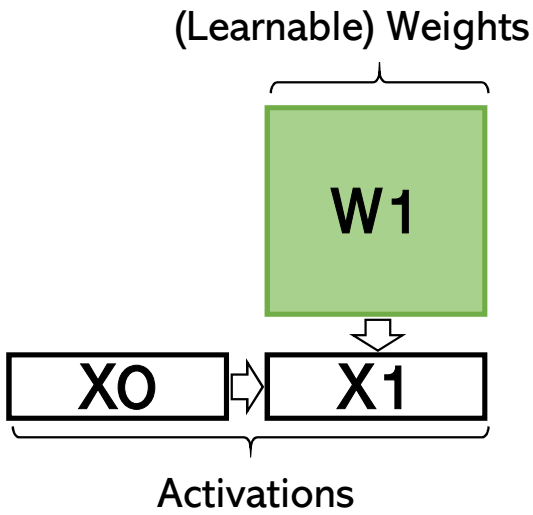
The LLM

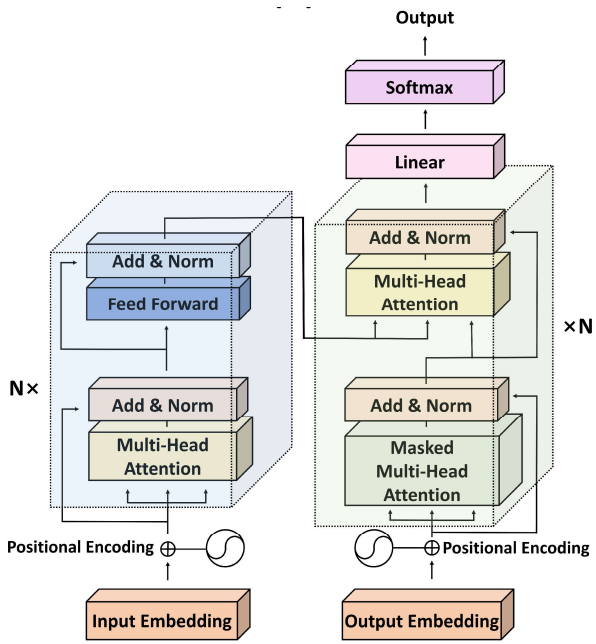
Let's represent an LLM layer as a matrix-matrix multiplication

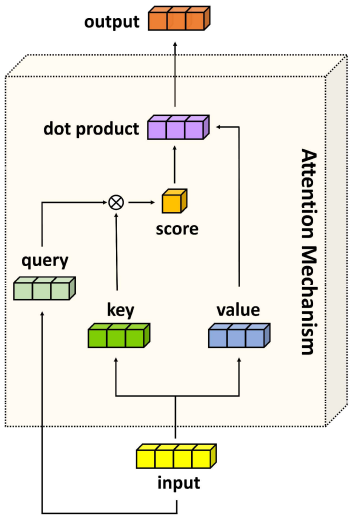


The LLM

Let's represent an LLM layer as a matrix-matrix multiplication

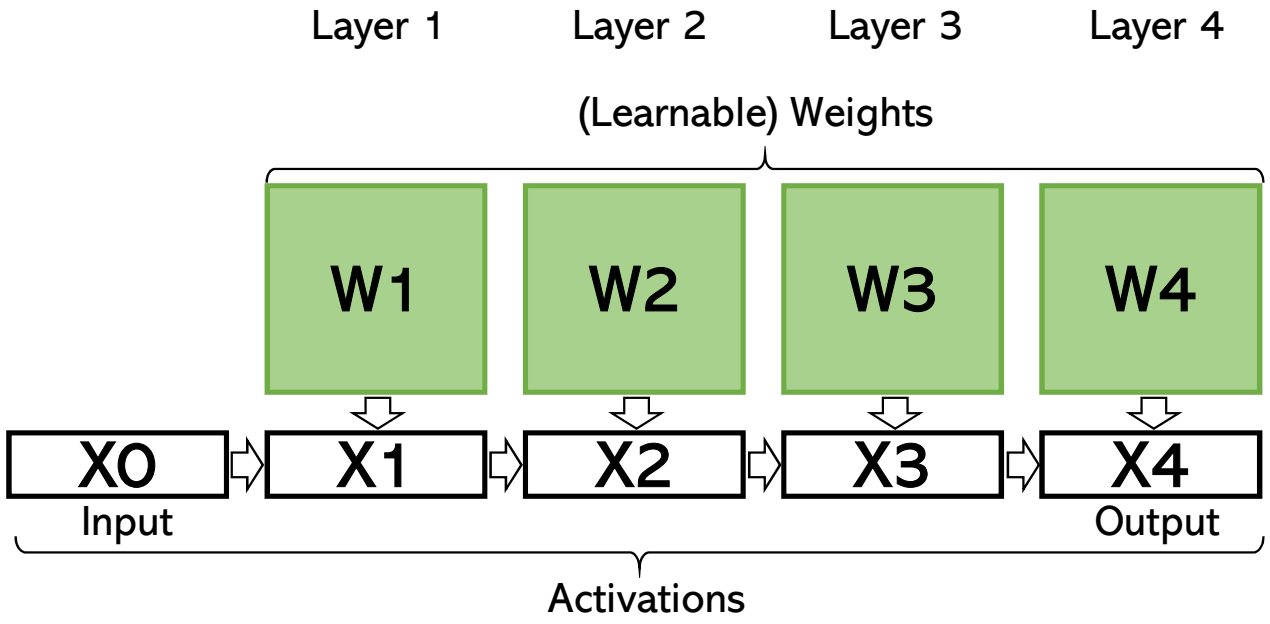






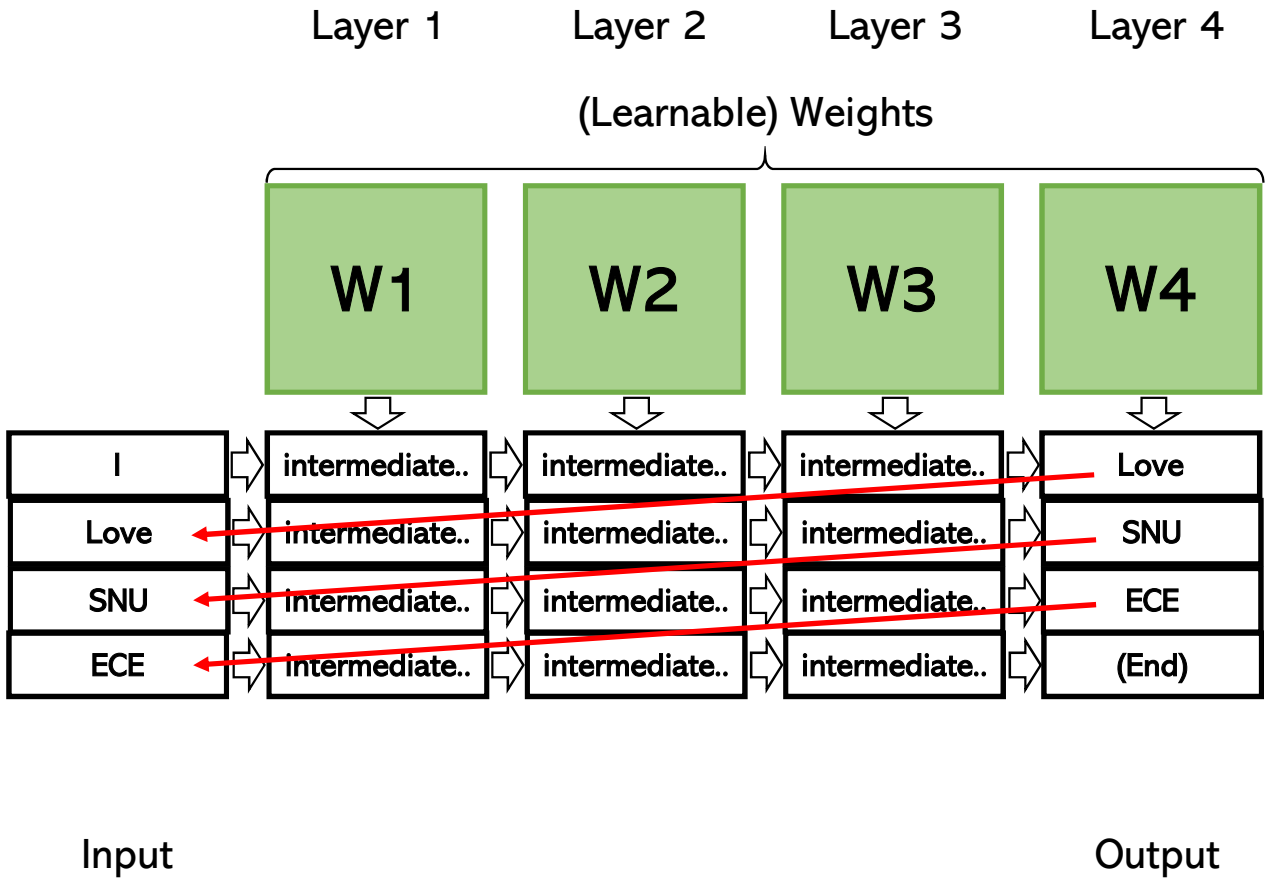
The LLM We Will Use For Now (Forward)

An example of a large language model with four layers

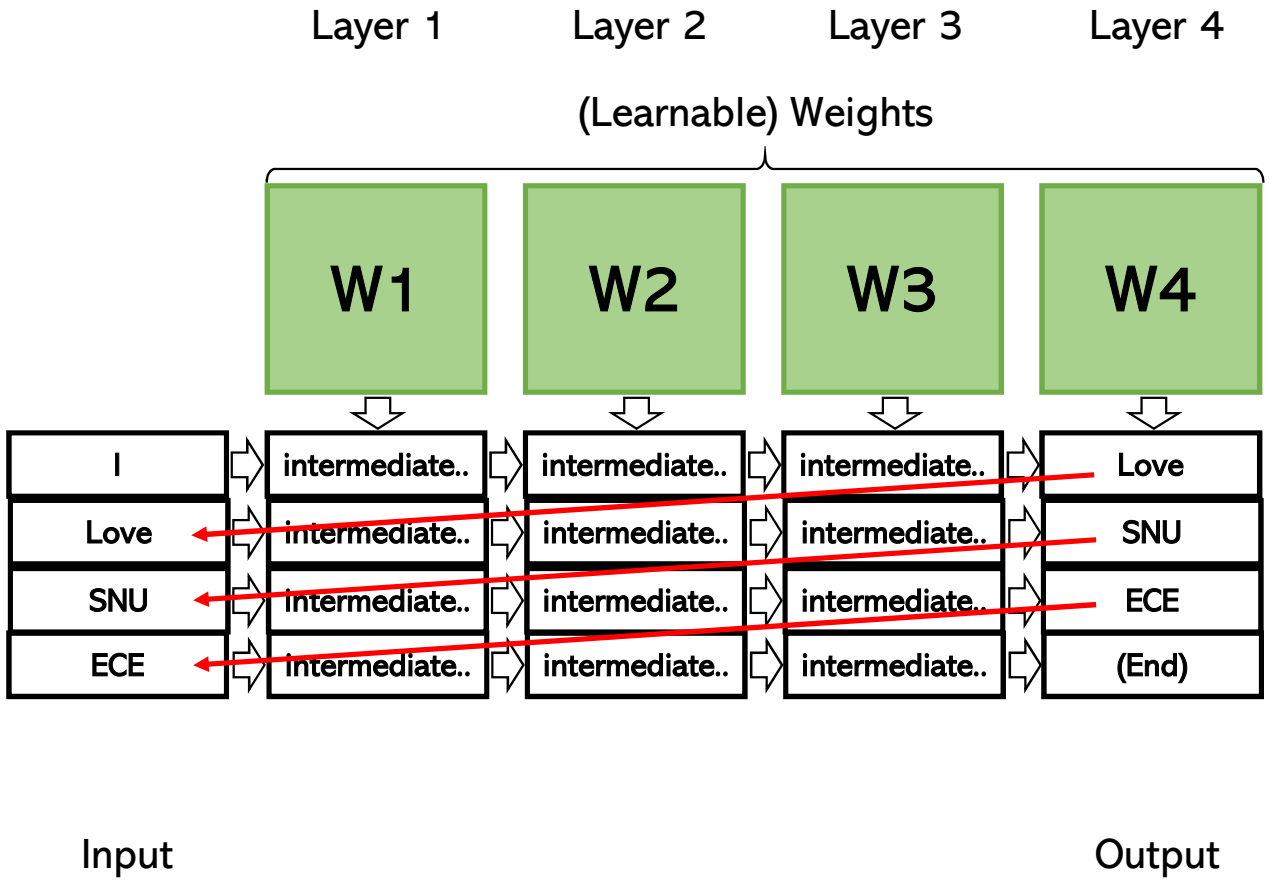


The LLM We Will Use For Now (Forward)

An example of a large language model with four layers (autoregressive)

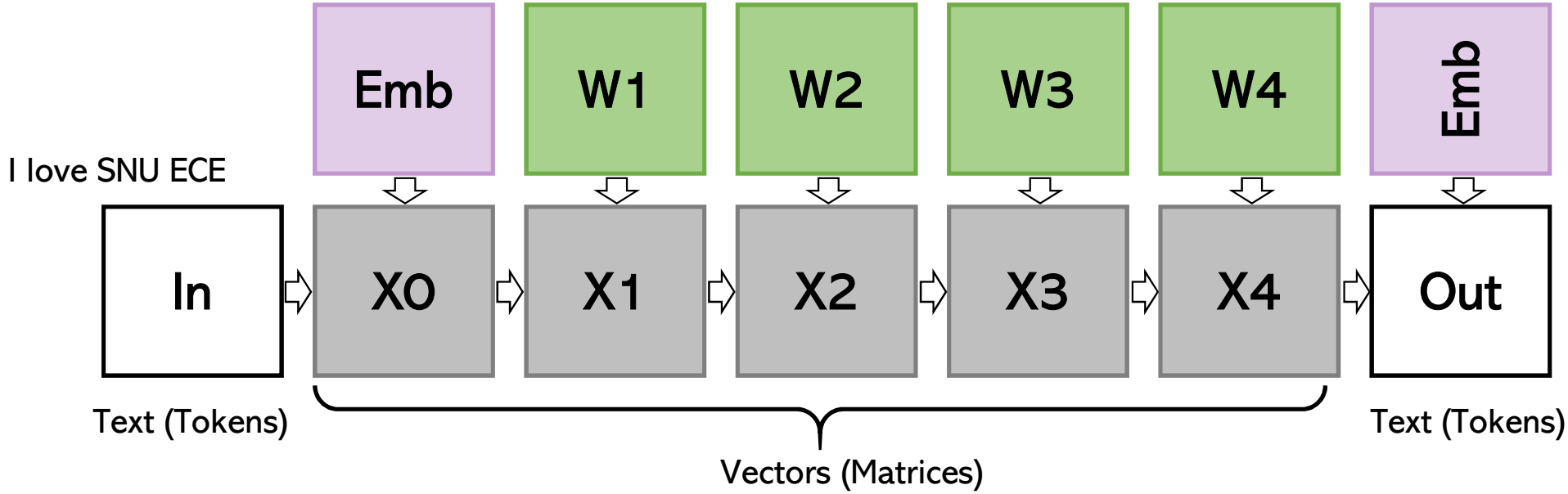


A Little Extra: How Is a Word a Vector?



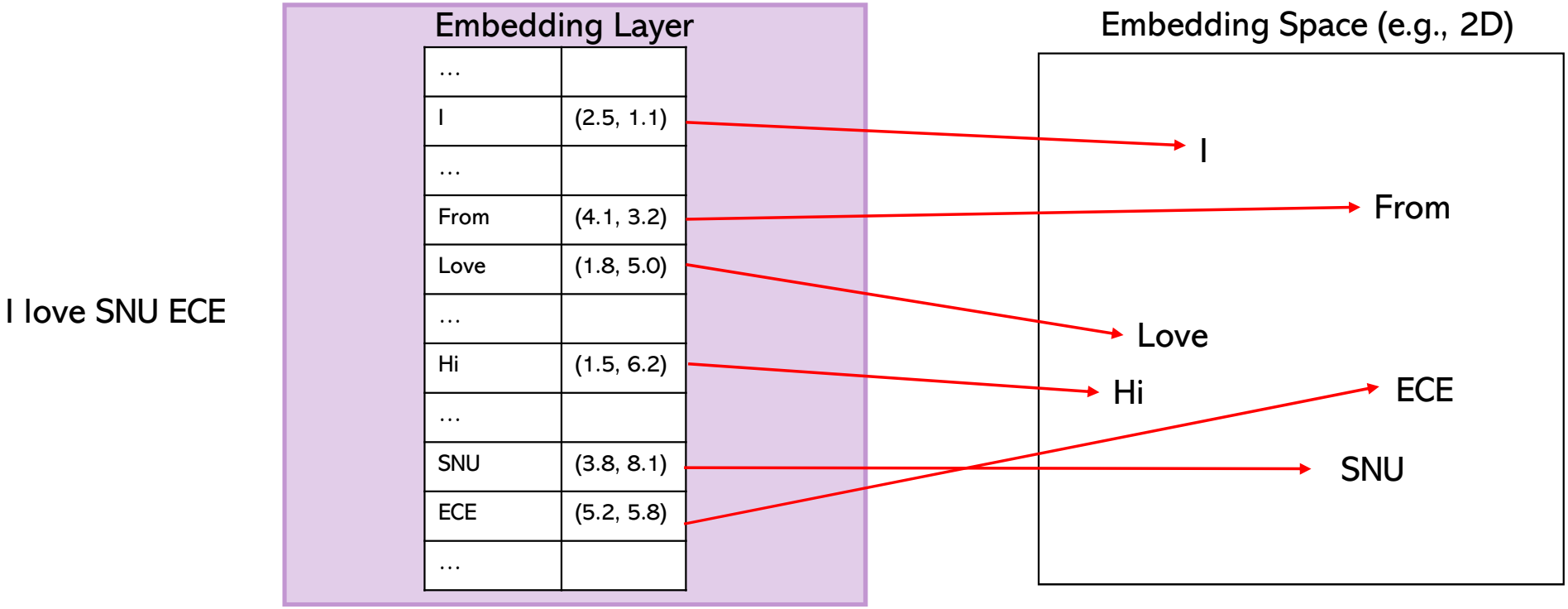
Embedding Layer

- Text is converted into vectors from "embedding"



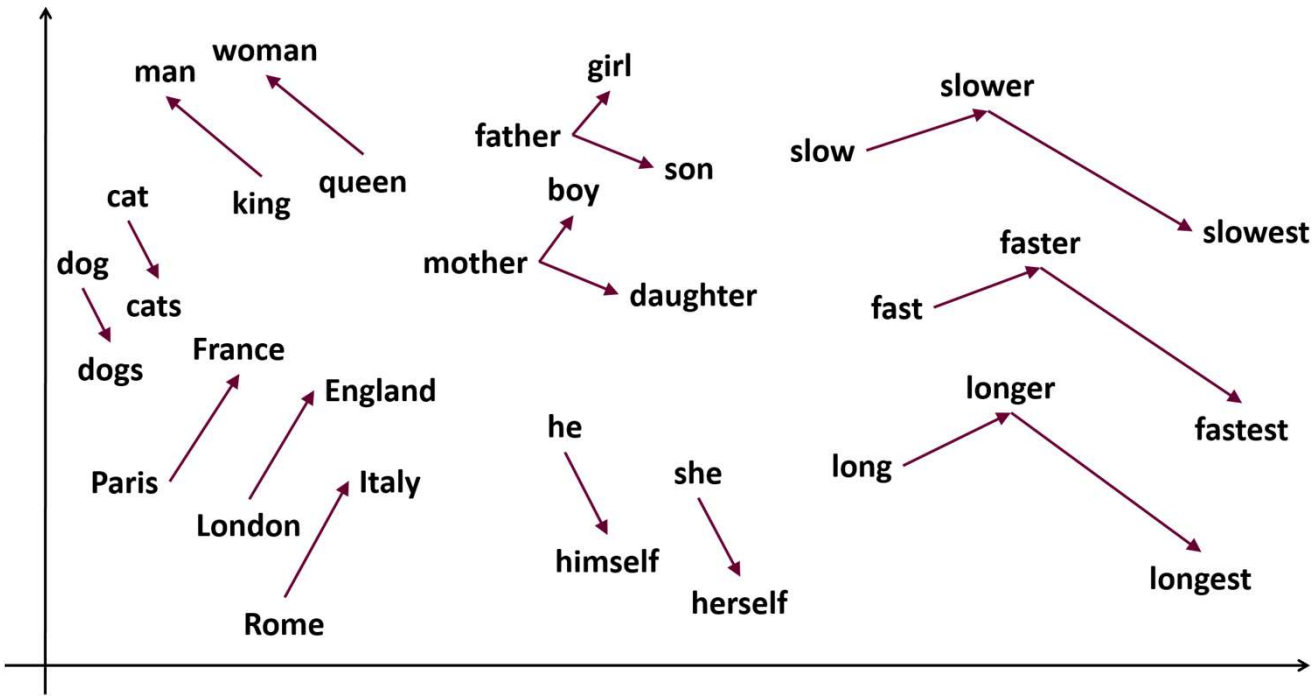
Retrieval: Search within Embedding Space

- Embedding maps contents to multi-dimensional space
  - Of 100~10000 dimensions



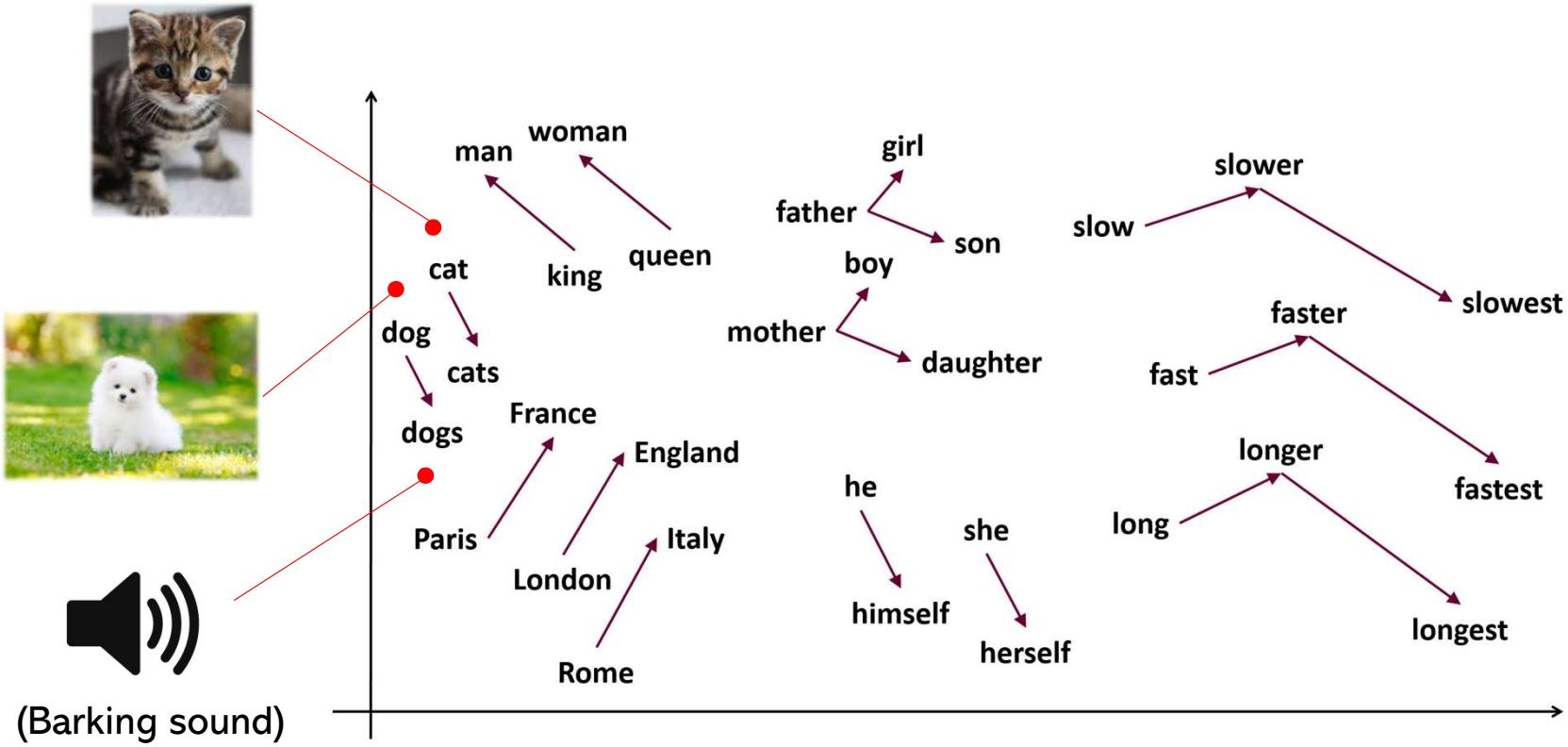
Embeddings Example: word2vec

- The locations in the embedding encode the meanings of words



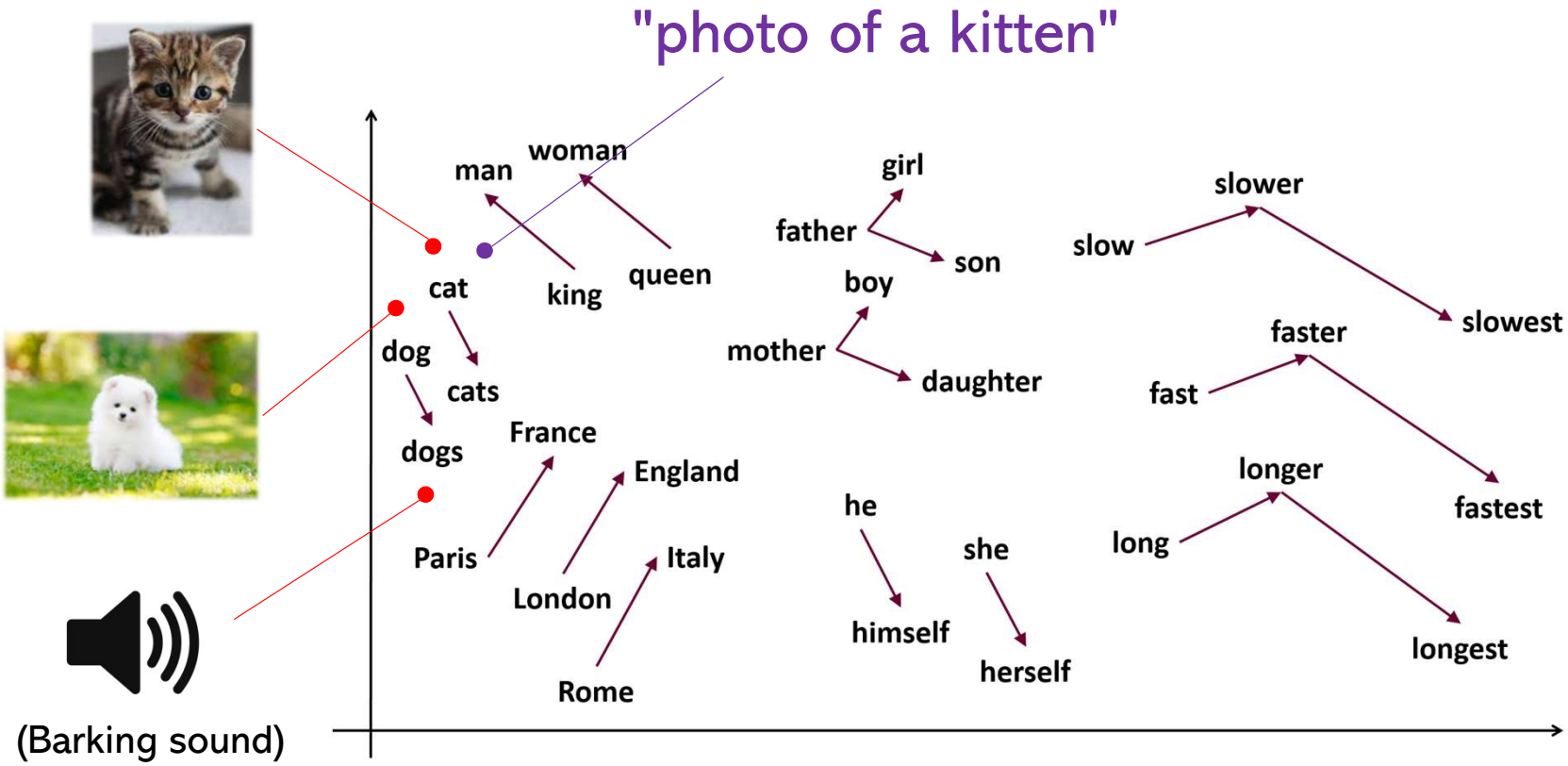
Embeddings Example: word2vec

- Embeddings can represent data with various modalities



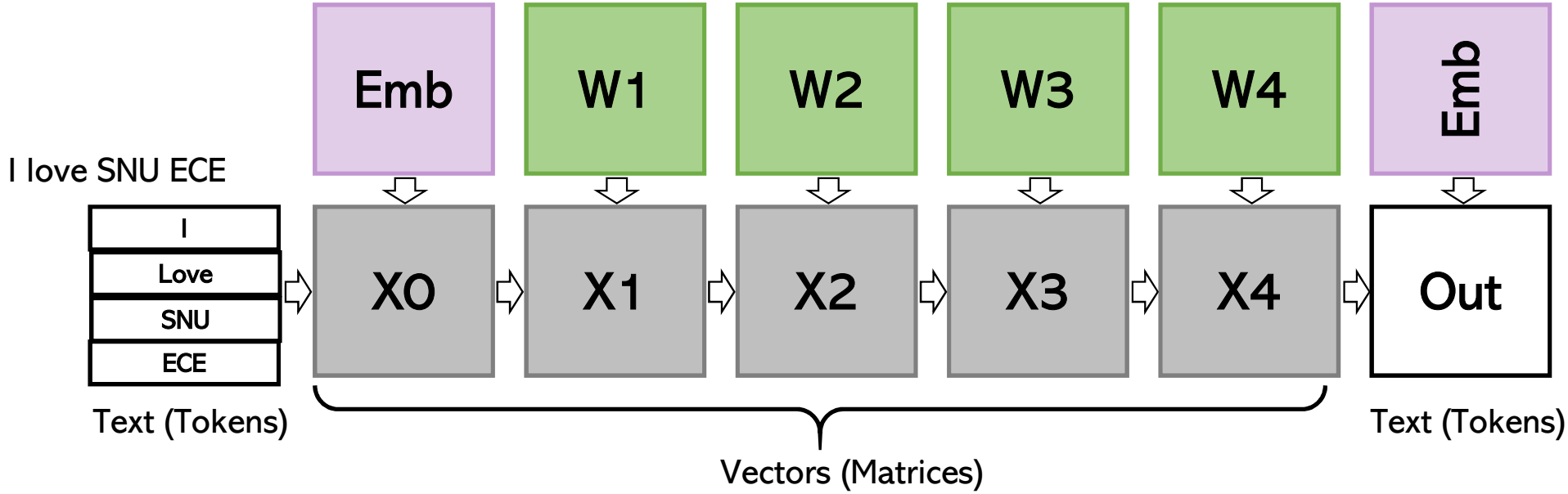
Embeddings Example: word2vec

- Locating the data in proximity is the key

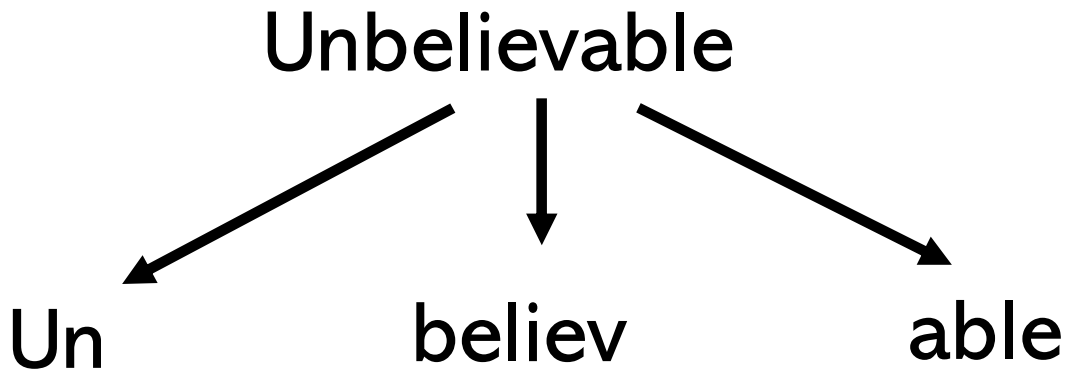


Tokens

- A token is not necessarily a word



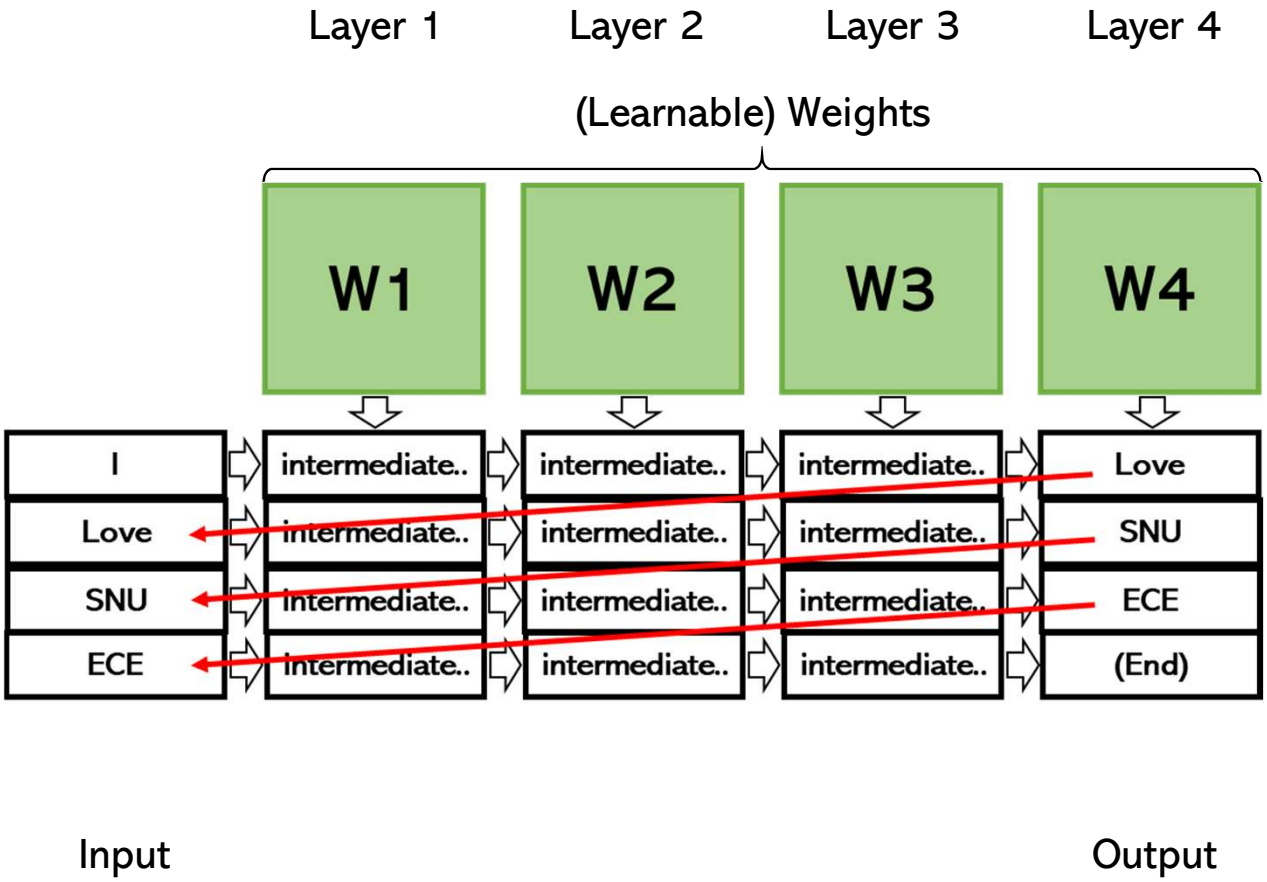
Tokenization



- BPE (Byte Pair Encoding) is commonly used
- → Merges the most frequently occurring adjacent pairs of character sequences into tokens
- Smaller vocabulary size – number of unique tokens are smaller than that of words
- Each token has its own morphological meaning

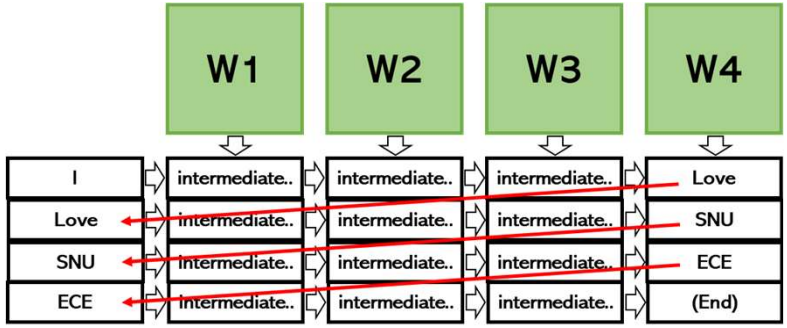
The LLM We Will Use For Now (Forward)

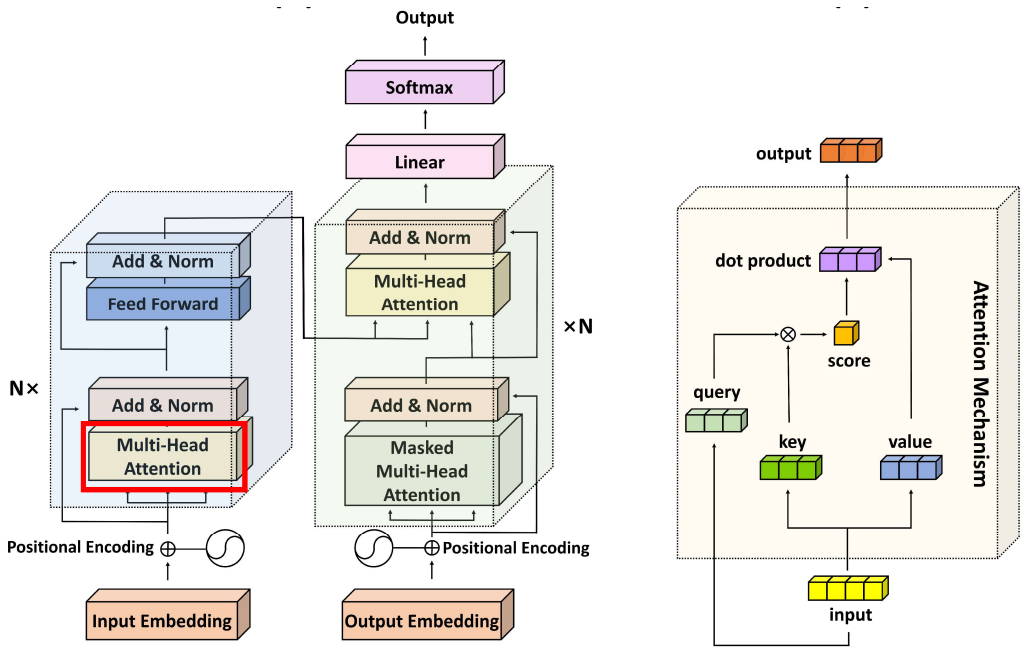
An example of a large language model with four layers (autoregressive)



The LLM in More Detail

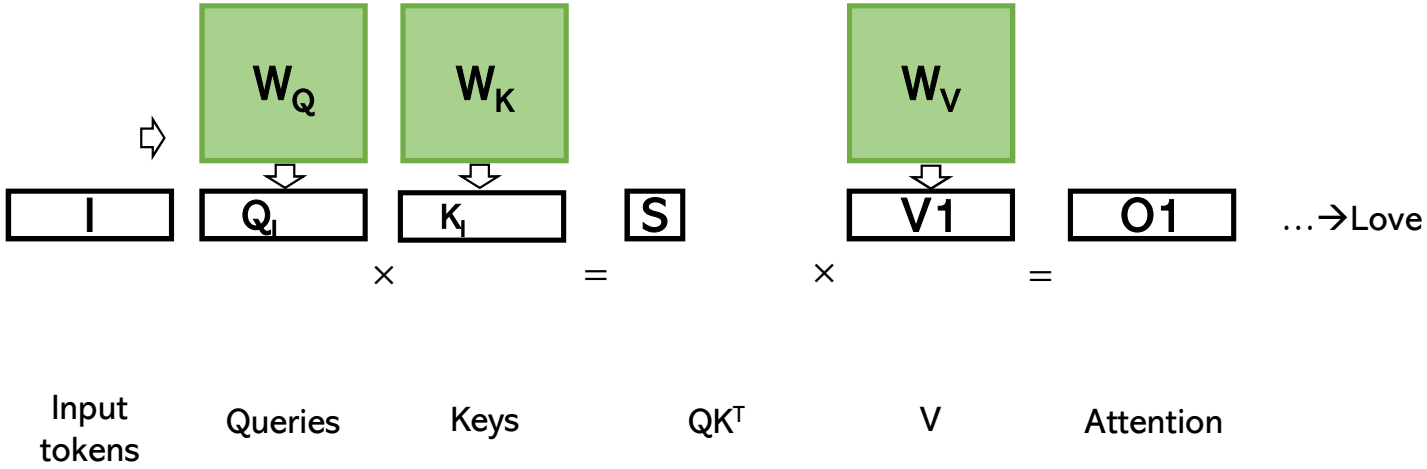
Multi-head Attention





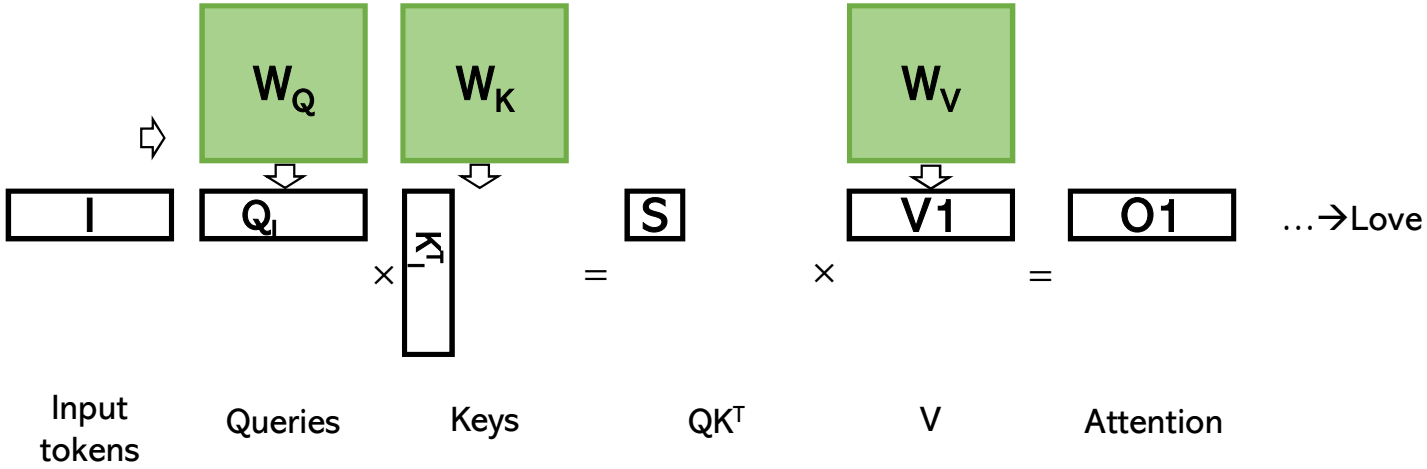
The LLM Inference

Multi-head Attention (Generating "I love SNU ECE")



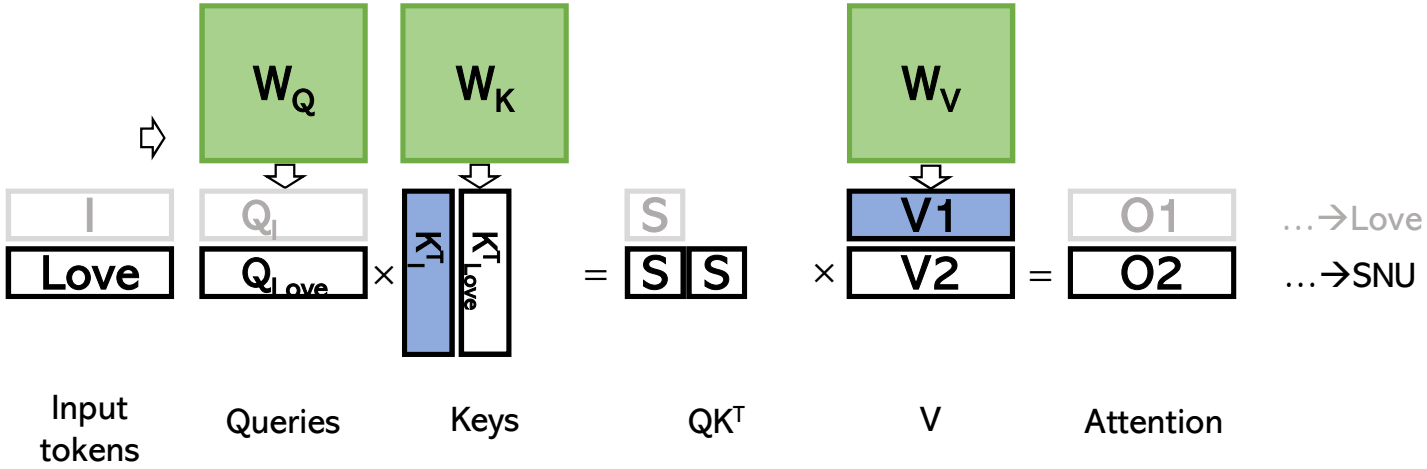
The LLM Inference

Multi-head Attention (Generating "I love SNU ECE")



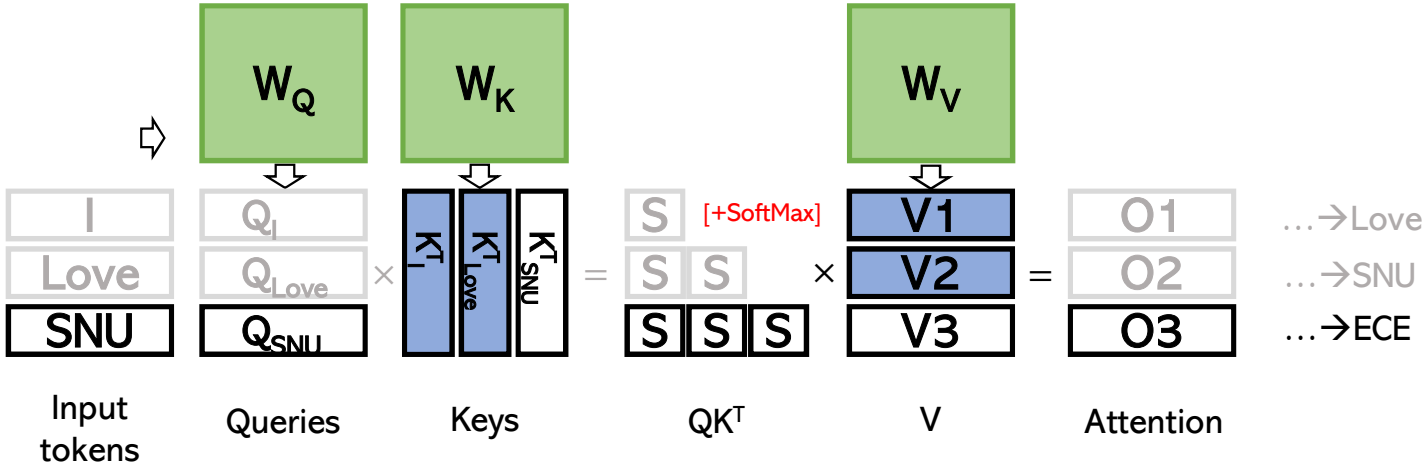
The LLM Inference

Multi-head Attention (Generating "I love SNU ECE")



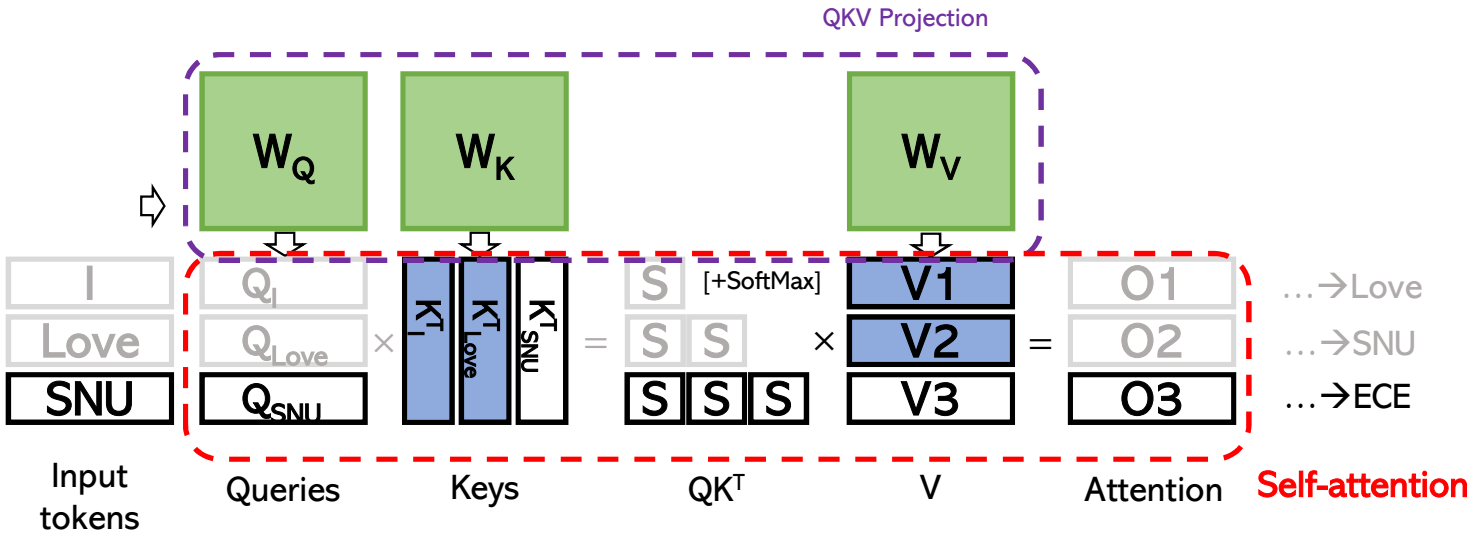
The LLM Inference

Multi-head Attention (Generating "I love SNU ECE")



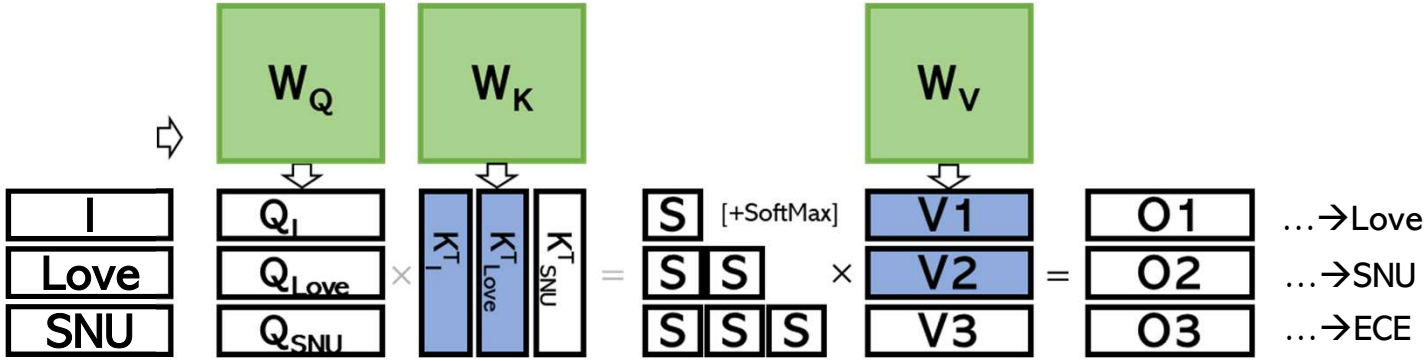
The LLM Inference

Multi-head Attention (Generating "I love SNU ECE")



The LLM Inference

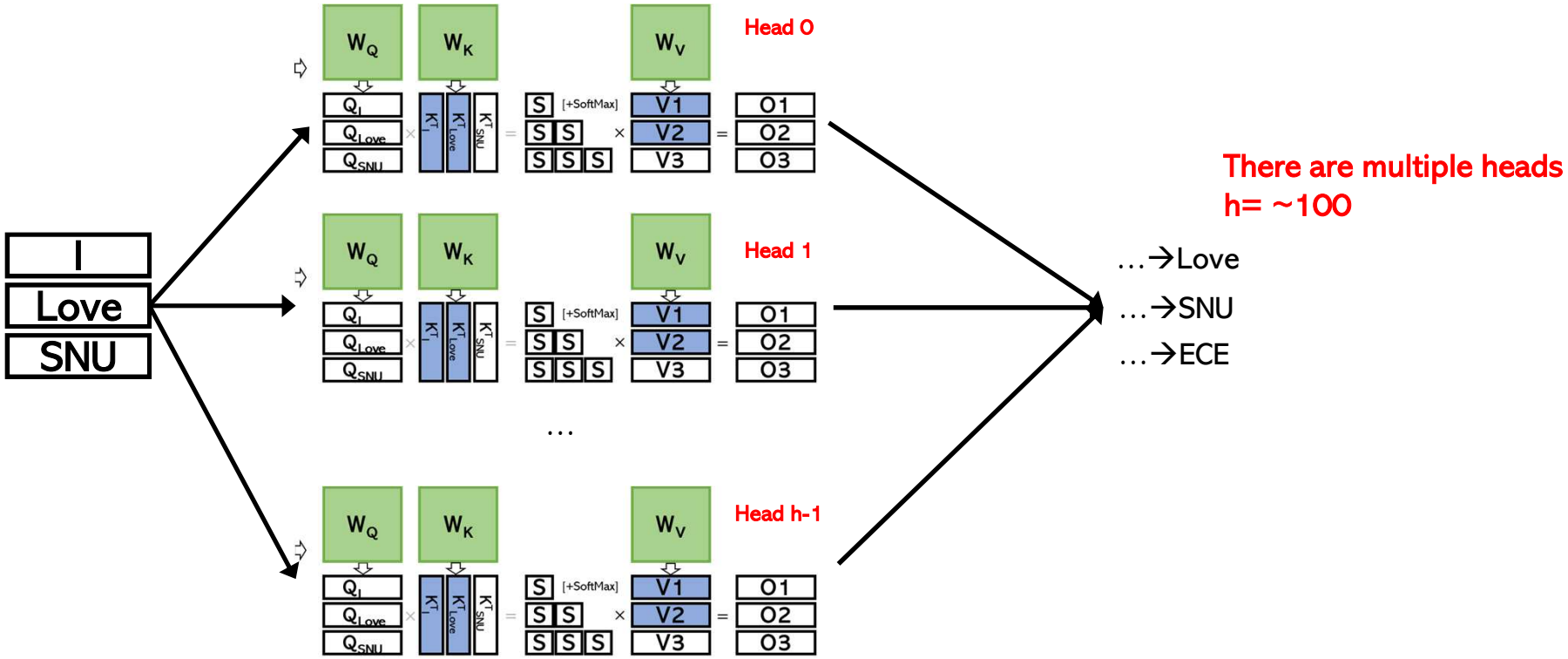
Multi-head Attention (Generating "I love SNU ECE")



An attention 'Head'

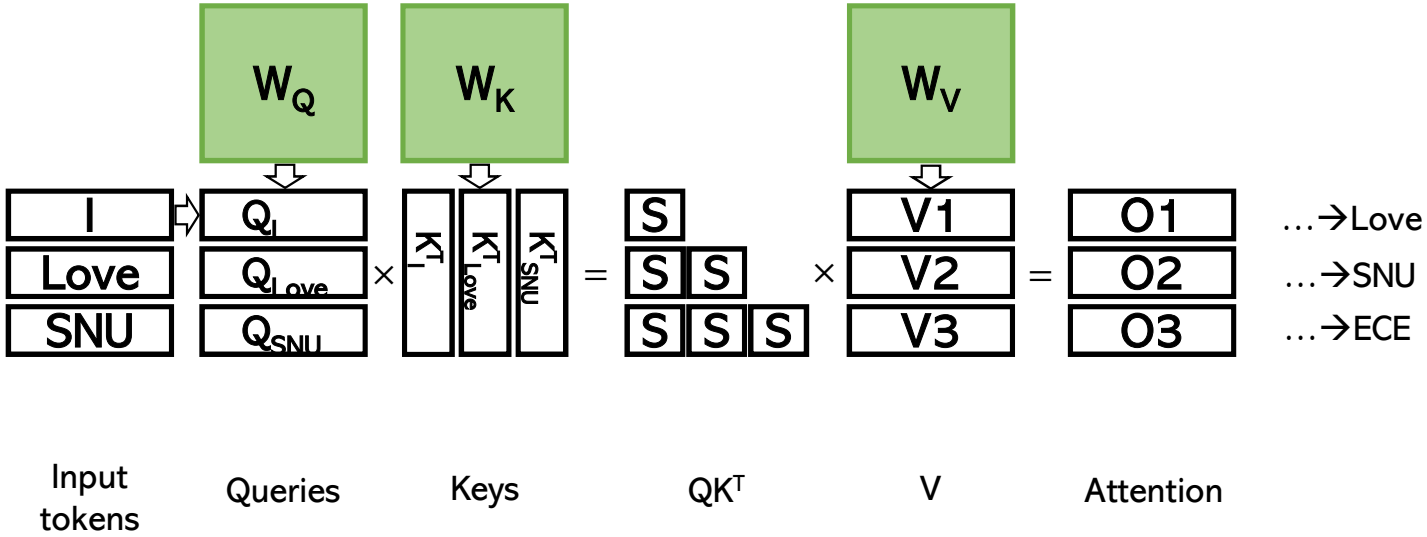
The LLM Inference

Multi-head Attention (Generating "I love SNU ECE")

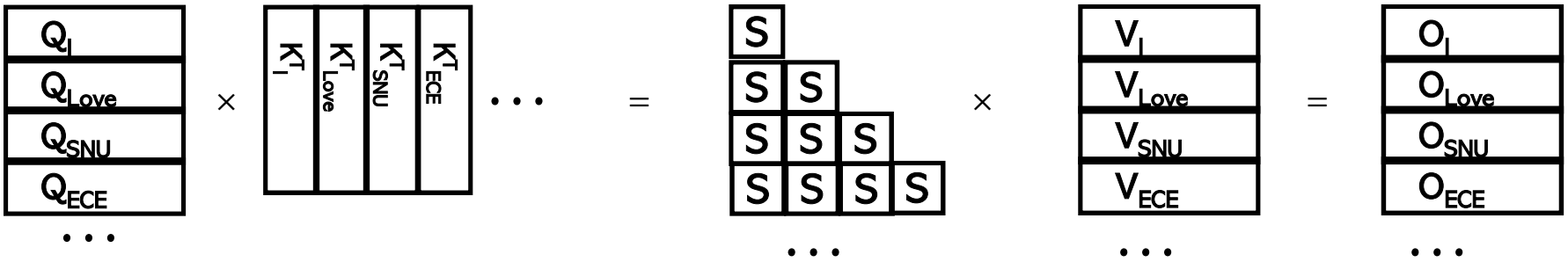


The LLM Inference

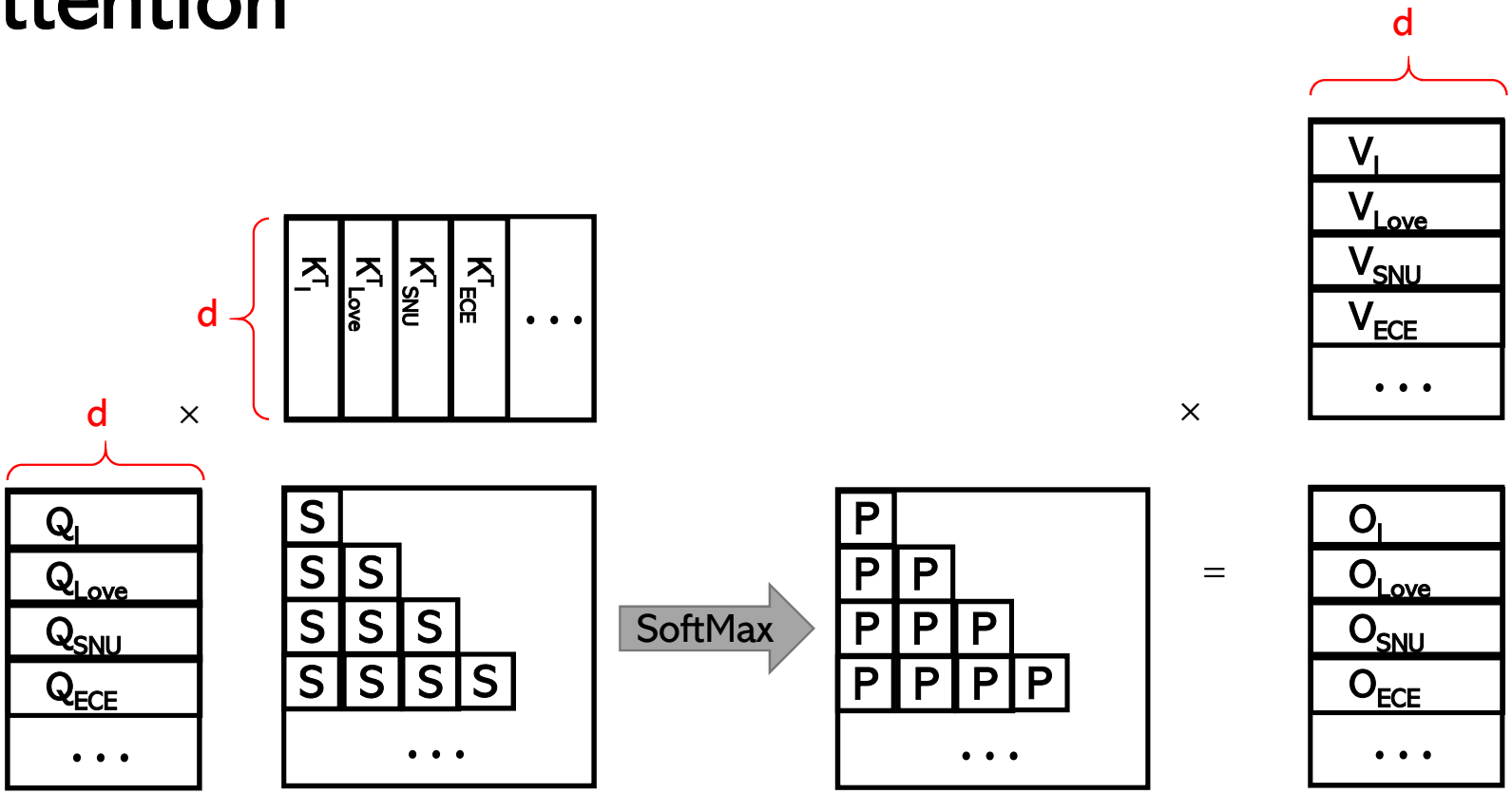
Multi-head Attention (Generating "I love SNU ECE")



Self-attention

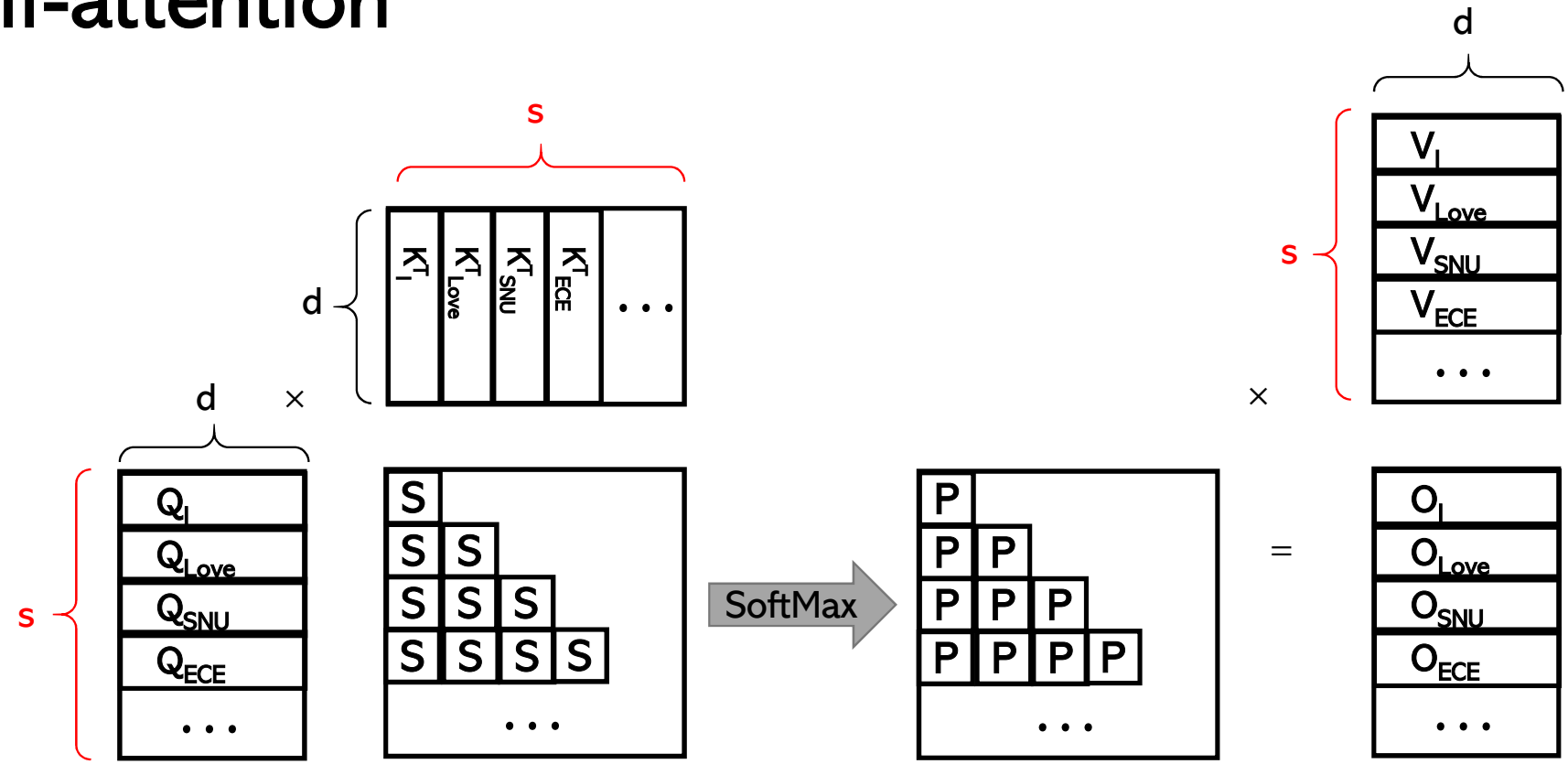


Self-attention



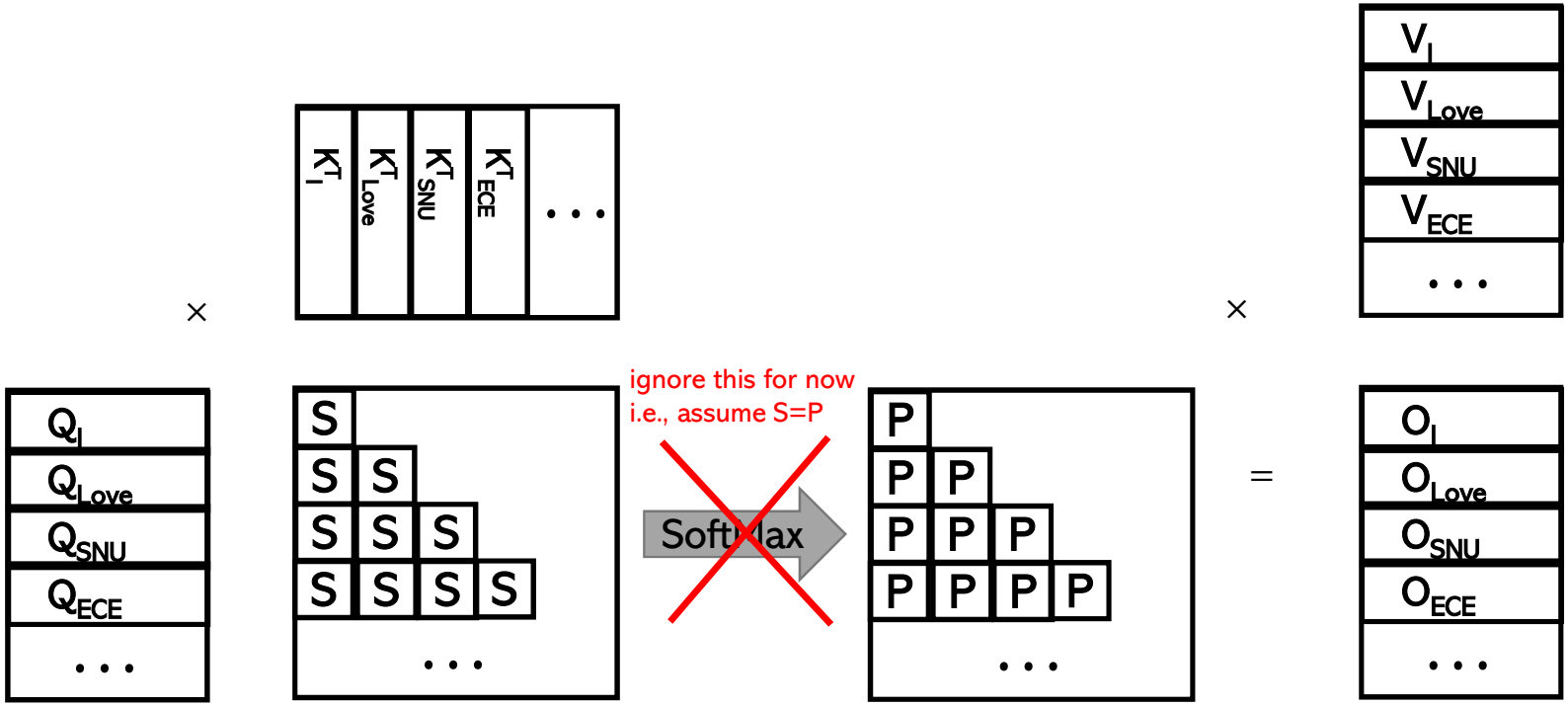
- d: head dimension. ~100

Self-attention



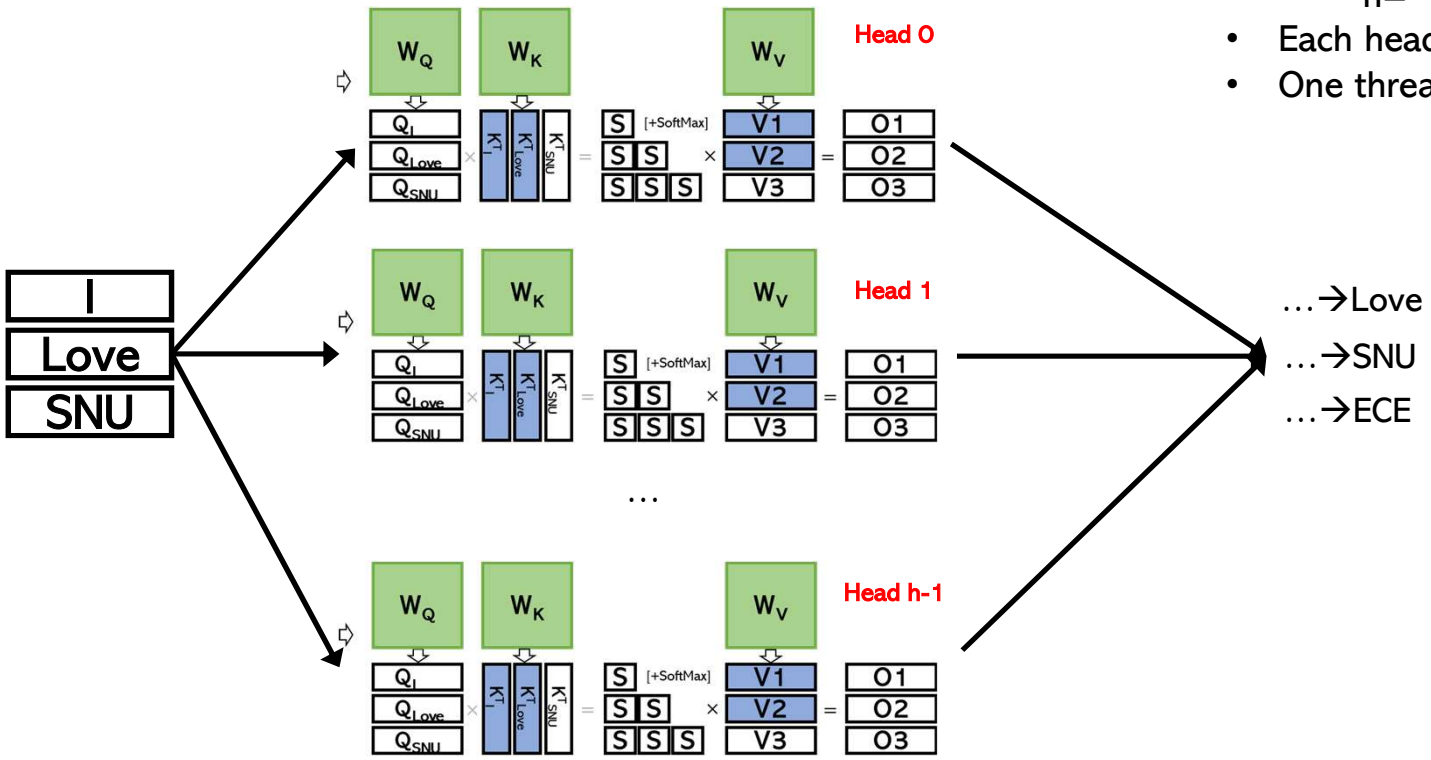
- d: head dimension. ~100
- s: sequence length: 1000 ~ 1,000,000 or more

Self-attention



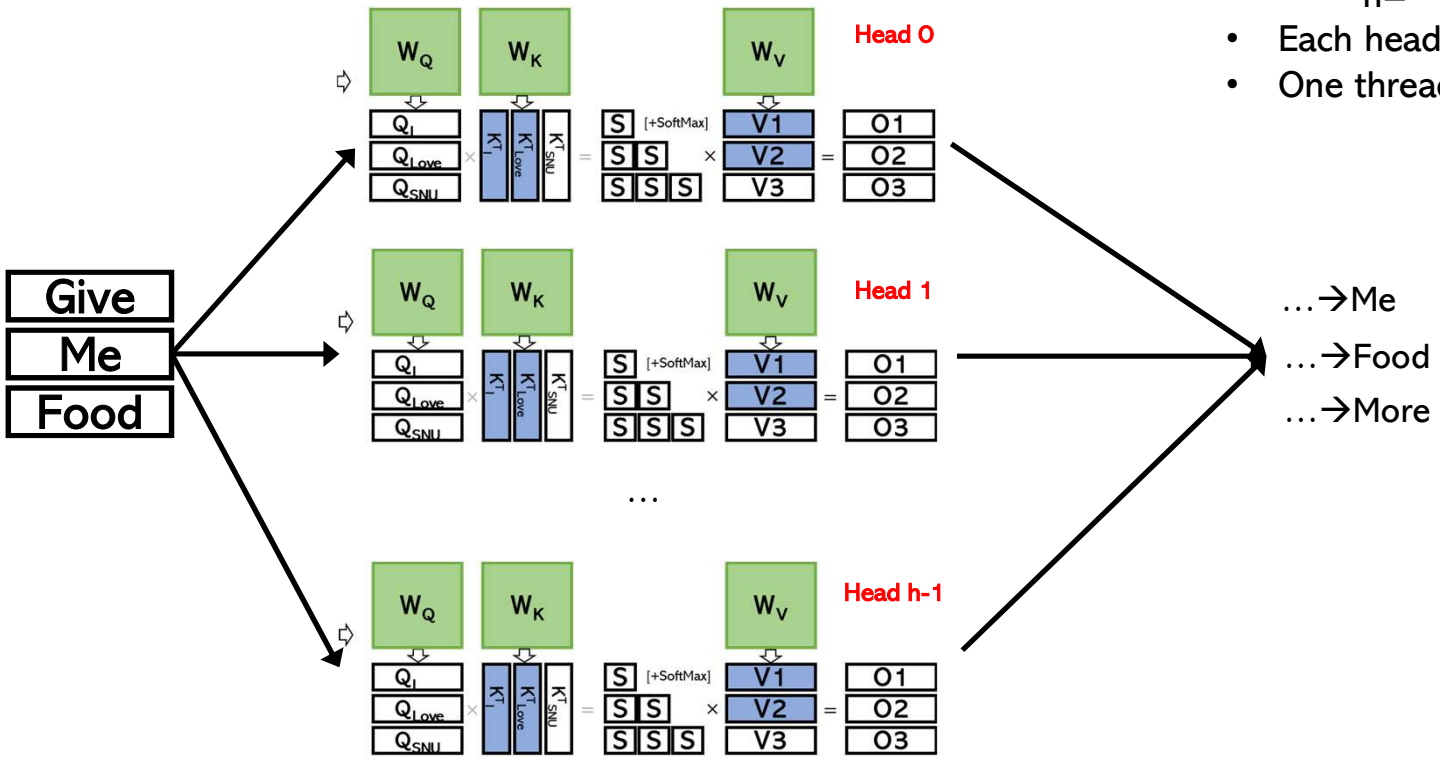
- d: head dimension. ~100
- s: sequence length: 1000 ~ 1,000,000 or more

Parallelization?



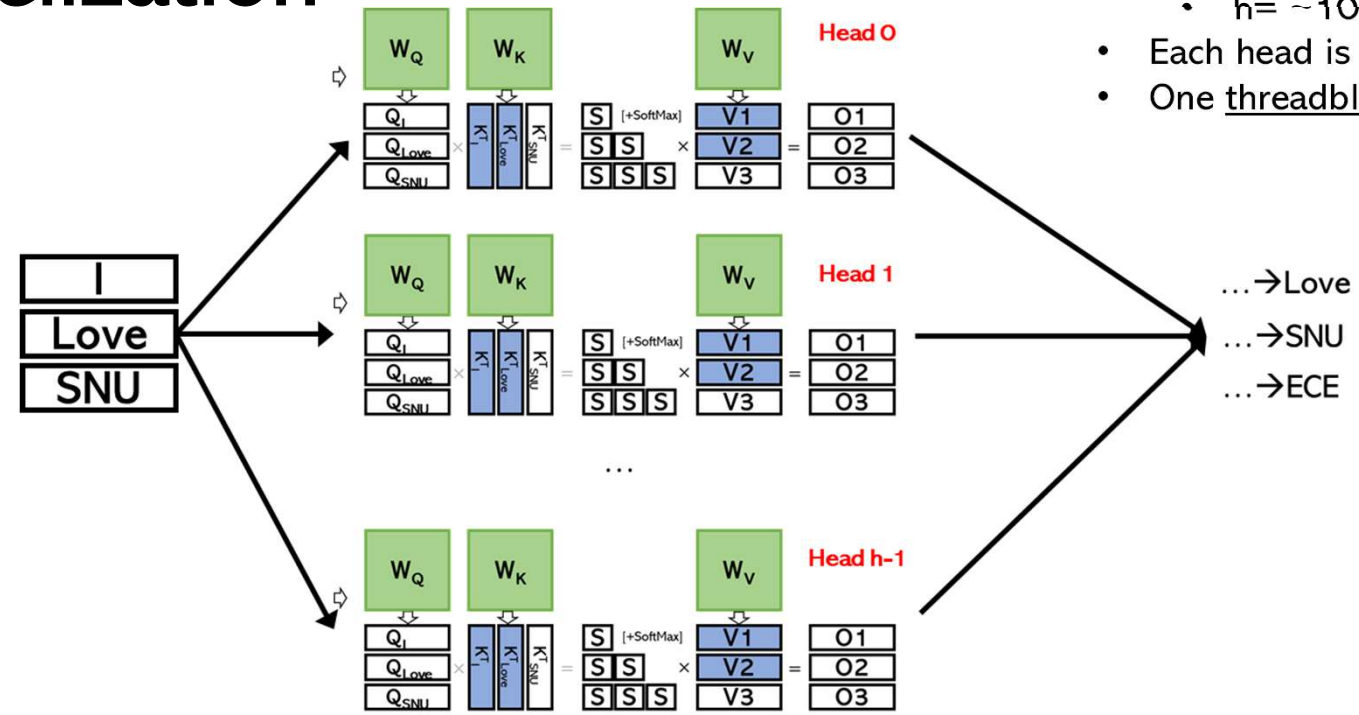
- There are multiple heads
  - h= ~100
- Each head is independent
- One threadblock per head

Parallelization?



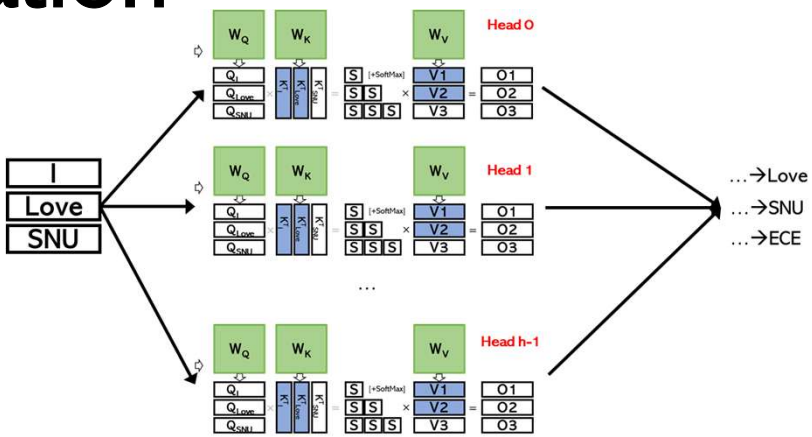
- There are multiple heads
  - h= ~100
- Each head is independent
- One threadblock per head

1. Parallelization



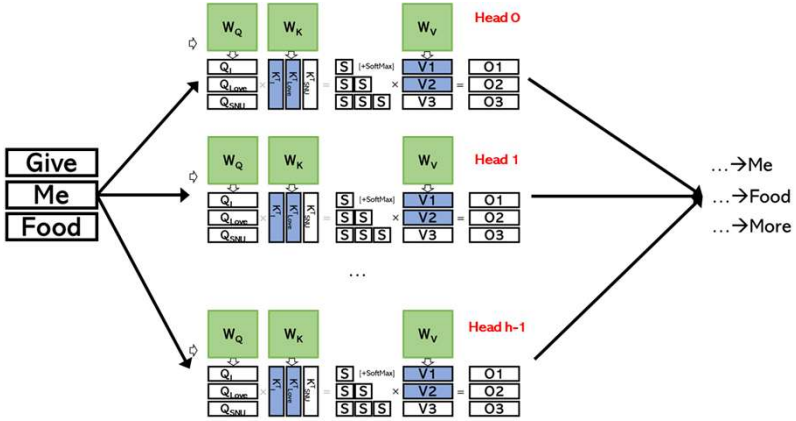
- There are multiple heads
  - h= ~100
- Each head is independent
- One threadblock per head

1. Parallelization



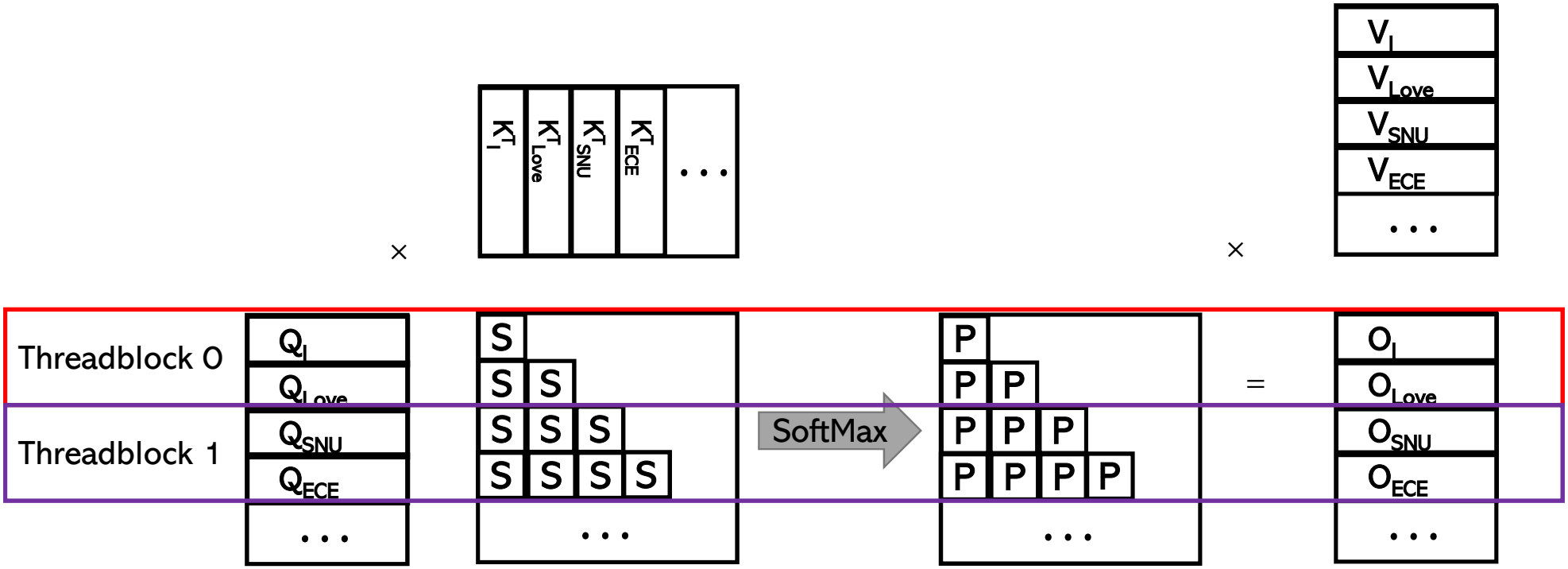
- There are multiple heads
  - h= ~100
- Each head is independent
- One threadblock per head

• • •



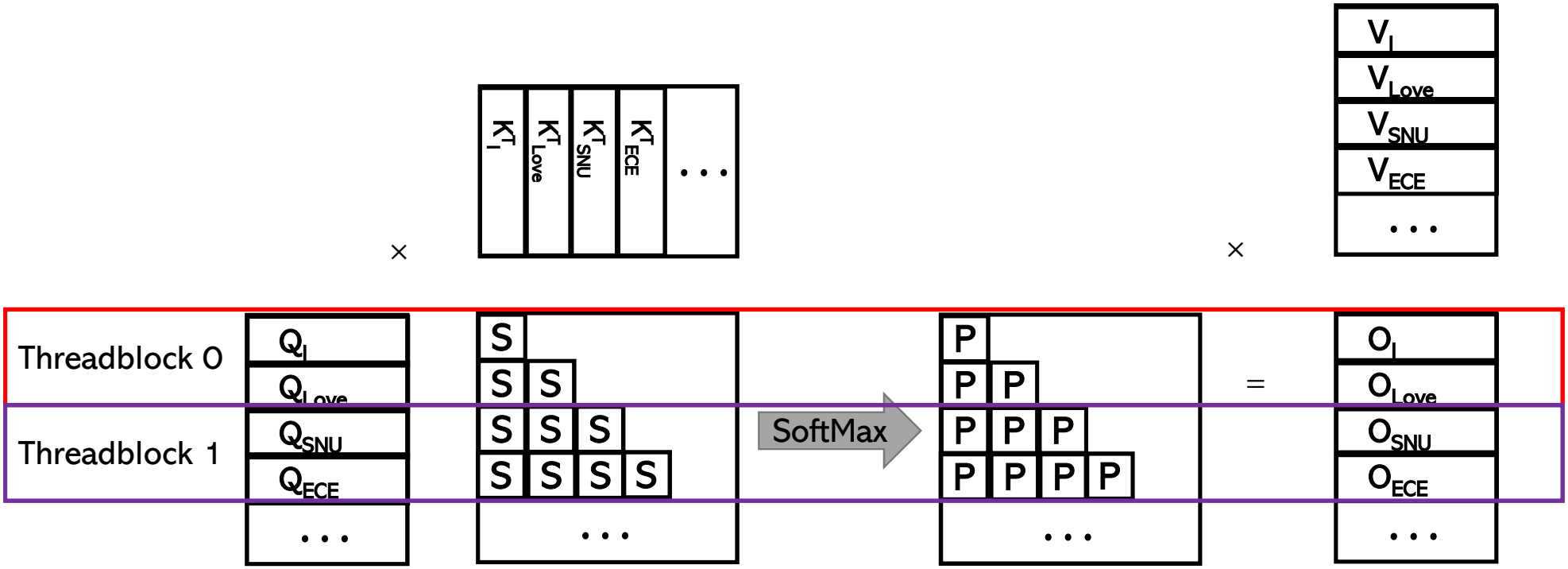
- If there are batches, you can use that too
- As many threadblocks as b*h
- b = 1~1024

1. Parallelization



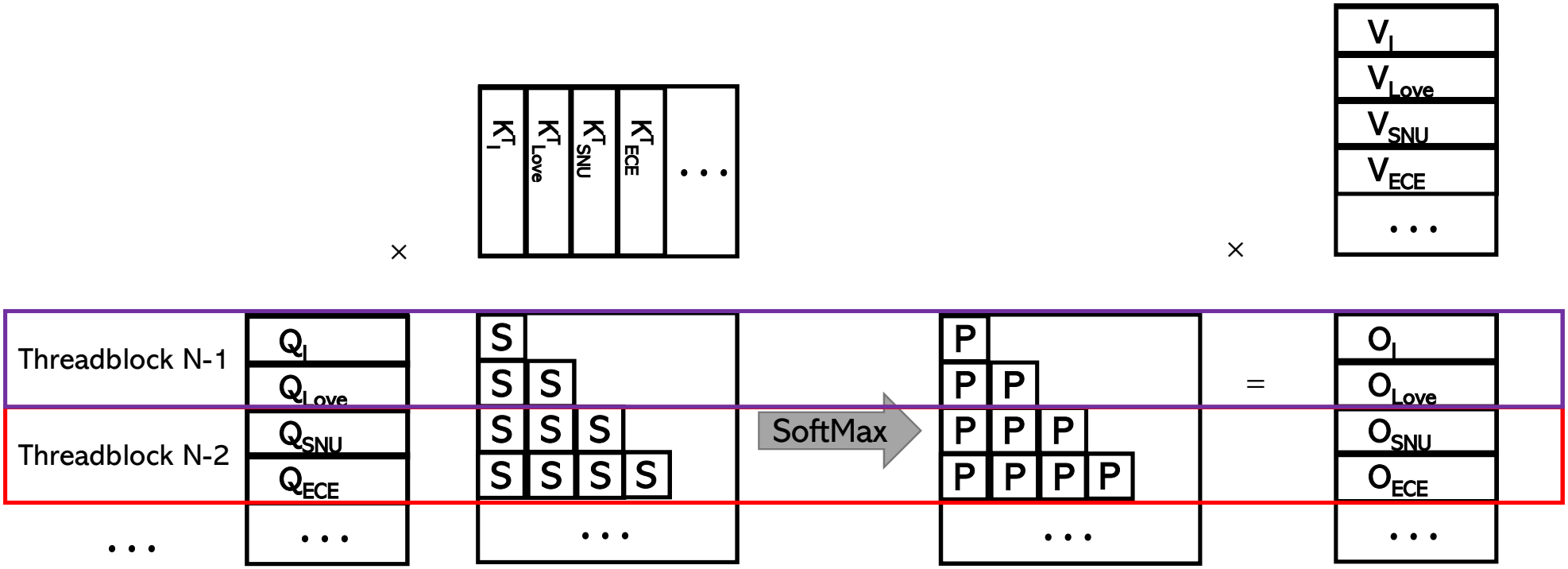
- Sometimes b*h is not large enough
  - There can be more than 100 SMs in a GPU
  - Sometimes b*h can be smaller than 100
- Use the sequence dimension for parallelization
- K, V have to be read multiple times

1. Parallelization – imbalance?



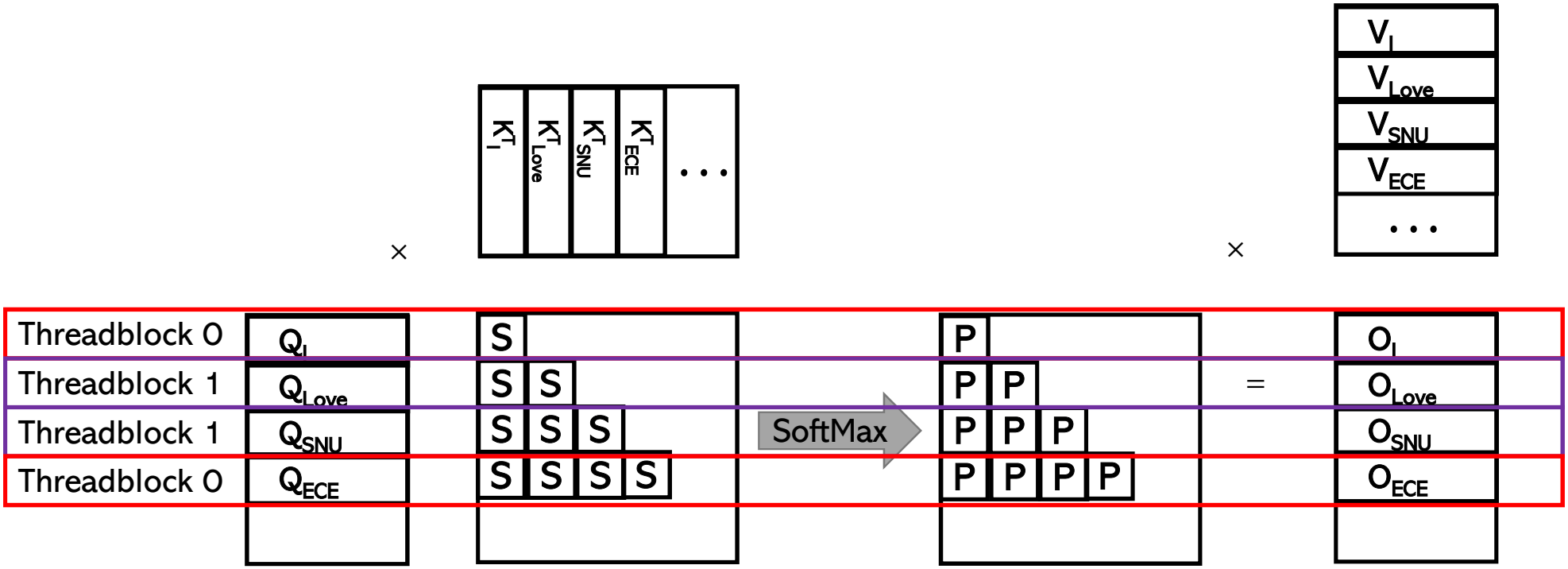
- Imbalance between threadblocks are automatically handled with dynamic threadblock assignment to SMs

1. Parallelization – imbalance?



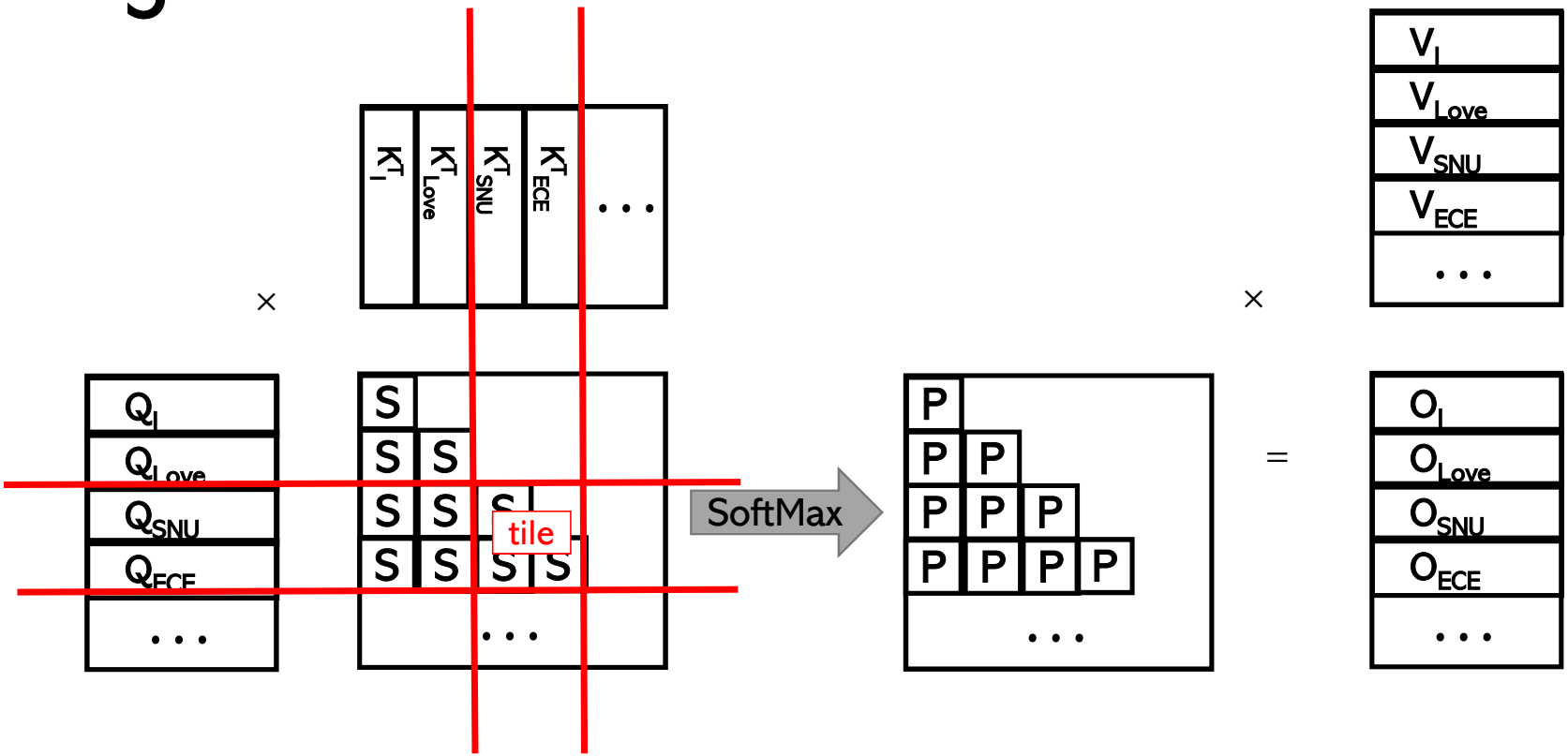
- Imbalance between threadblocks are automatically handled with dynamic threadblock assignment to SMs
- Reverse order can be better for load imbalances

1. Parallelization – imbalance?



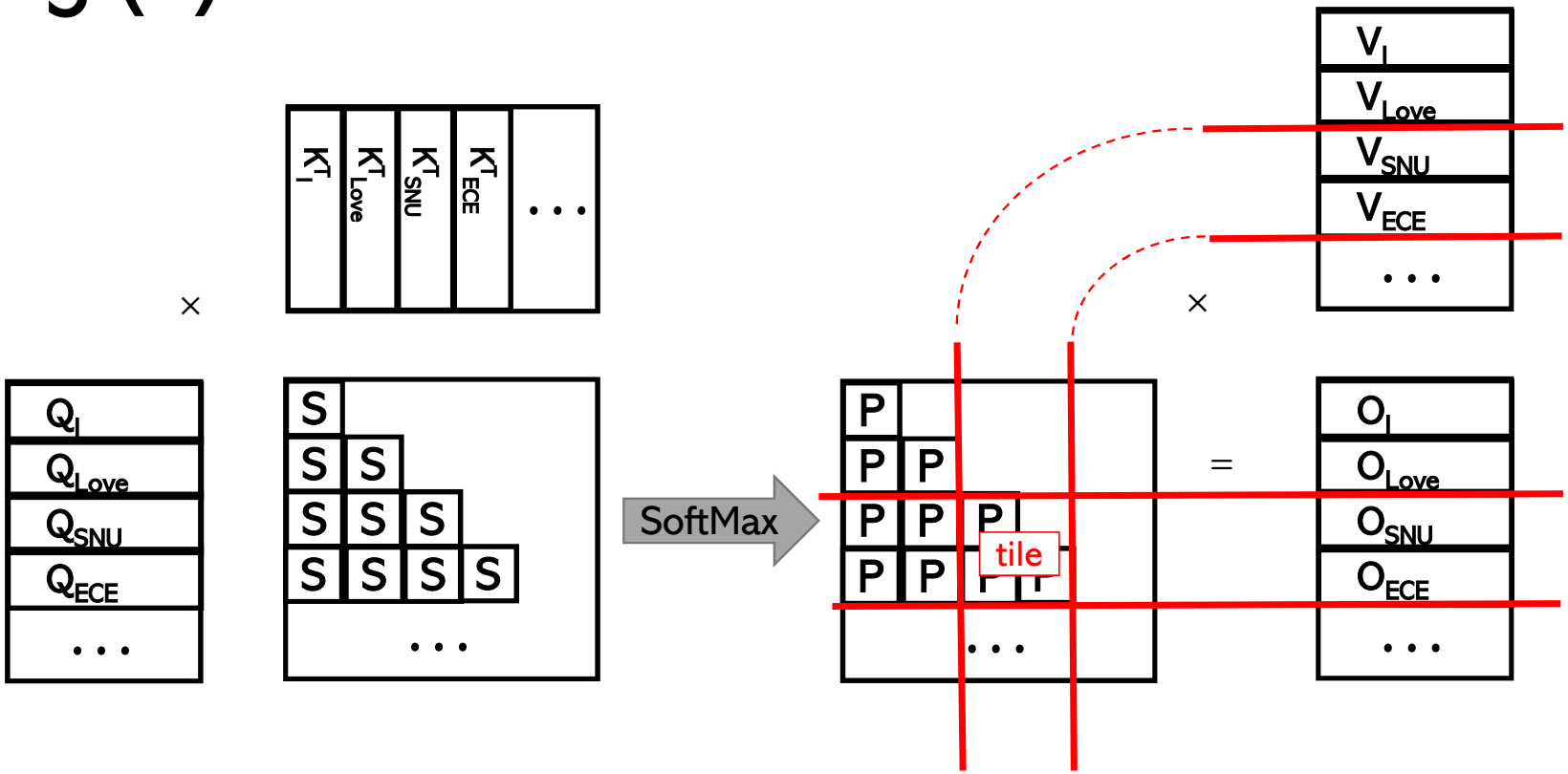
- ZigZag scheduling is also possible

2. Tiling



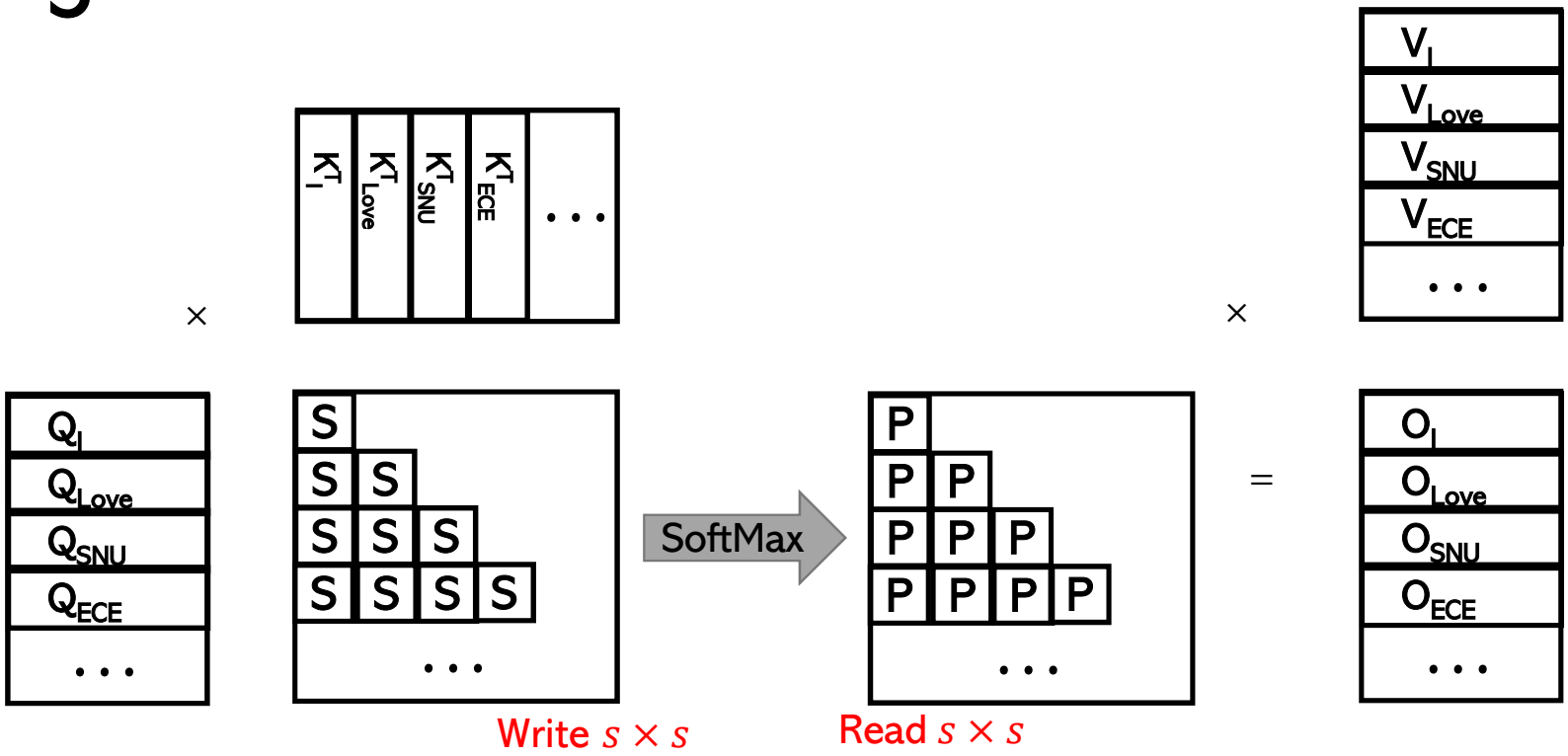
- Entire Q, K, S in global memory
- Each tile in shared memory

2. Tiling (2)



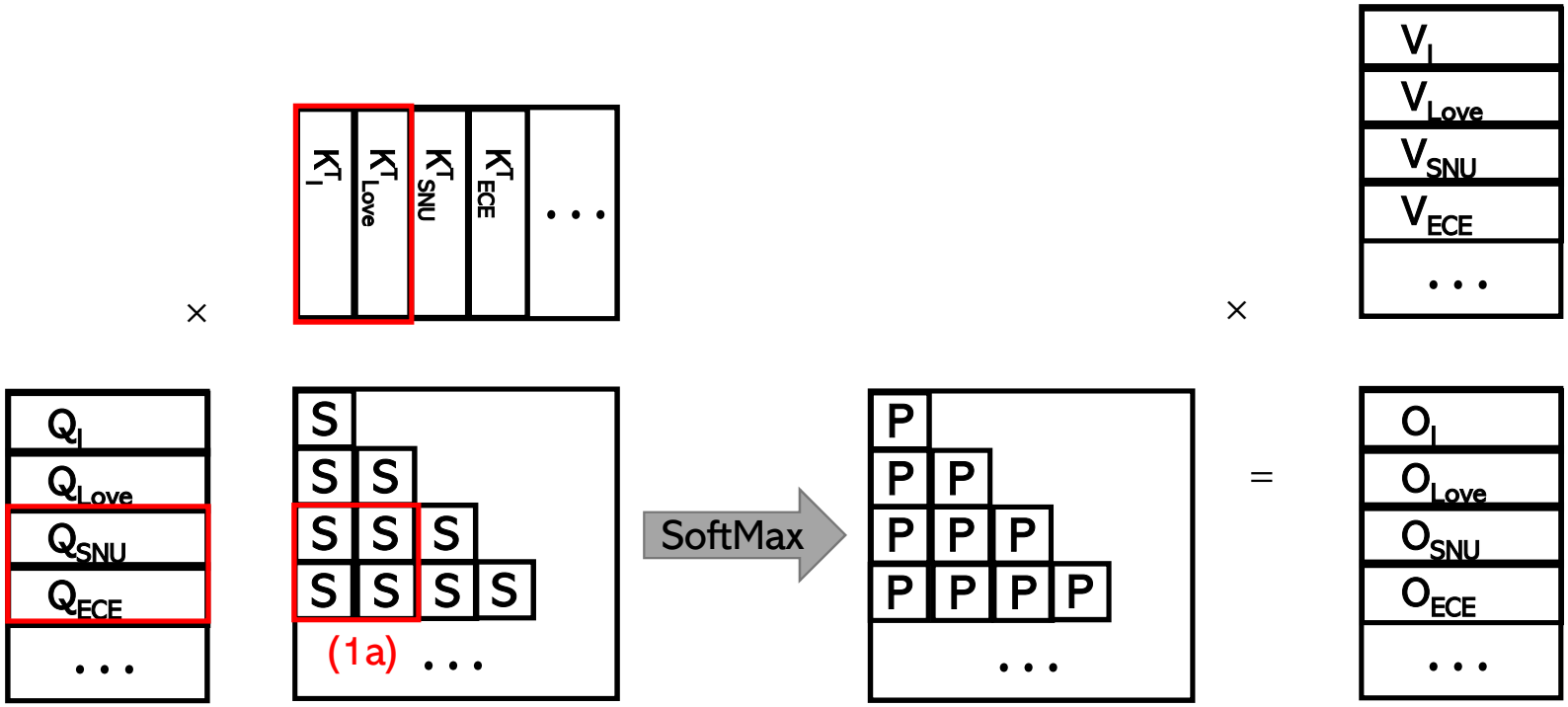
- Entire P, V, O in global memory
- Each tile in shared memory
- Apply tiling at the input P because it's larger

2. Tiling - issues



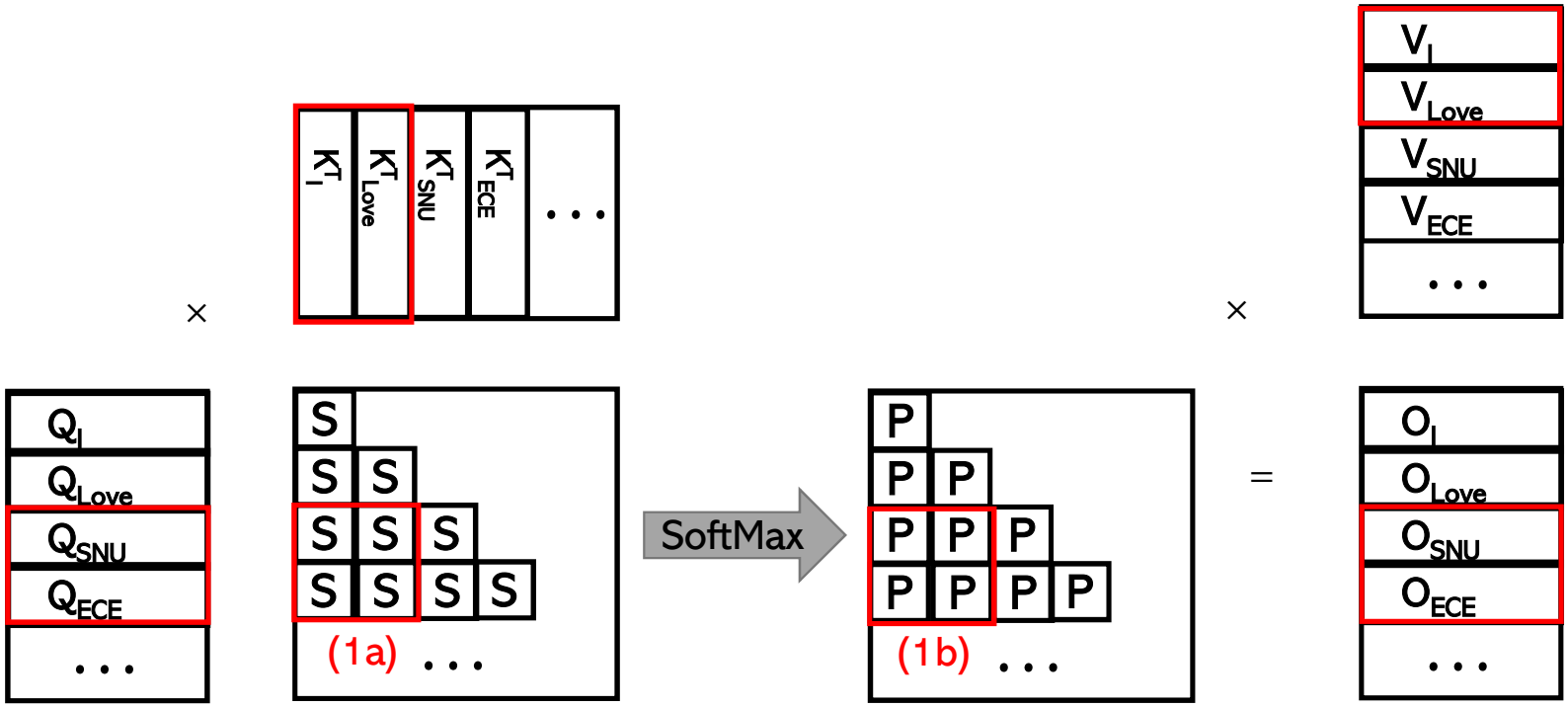
- S and P are the largest matrices with $s \times s$ size.
- S is written to the global memory, just to be read again
  → large memory bandwidth usage

3. Kernel Fusion



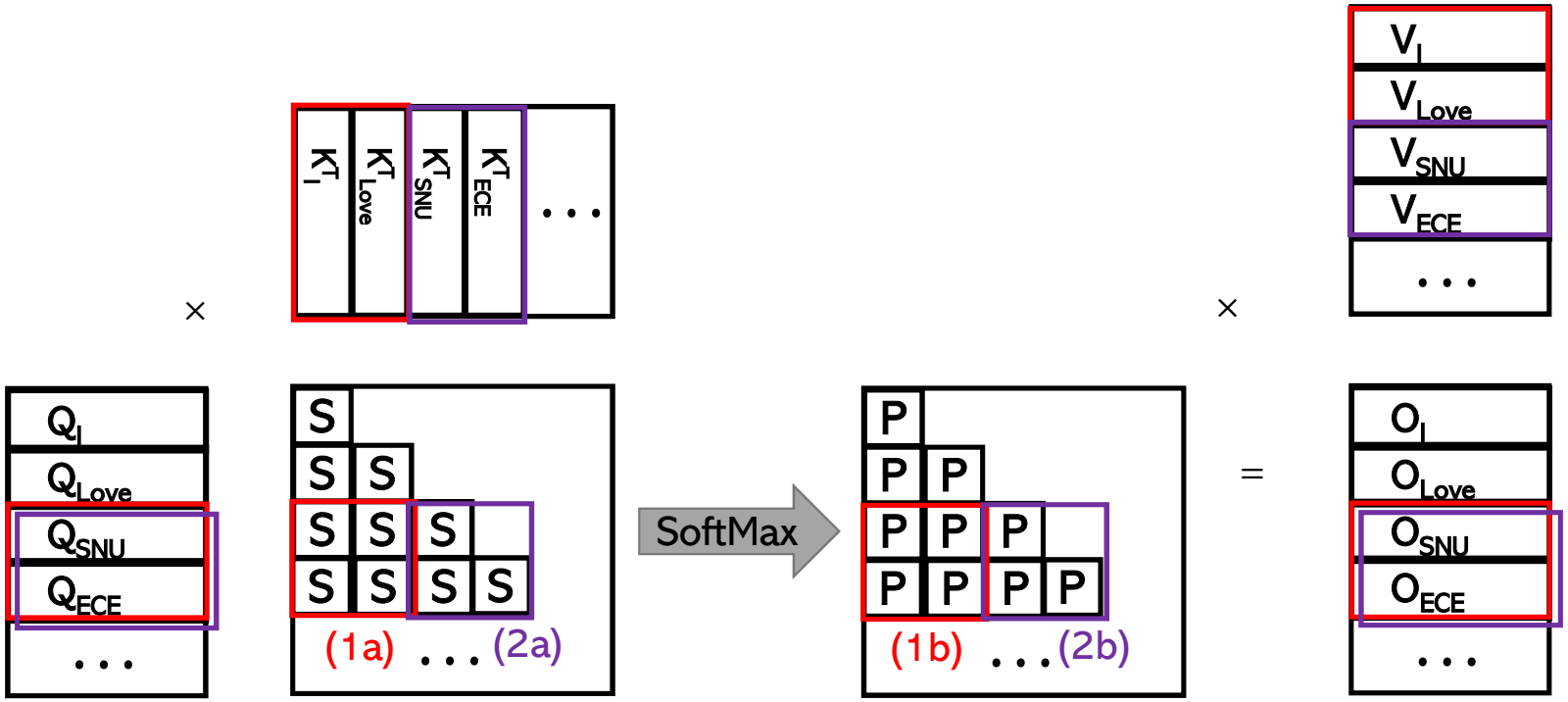
- Compute a block for S
- Use that block to compute a block of O

3. Kernel Fusion



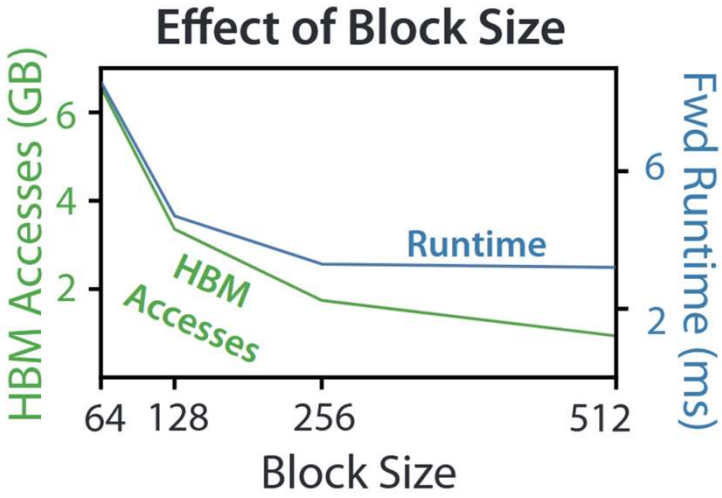
- Compute a block for S
- Use that block to compute a block of O

3. Kernel Fusion



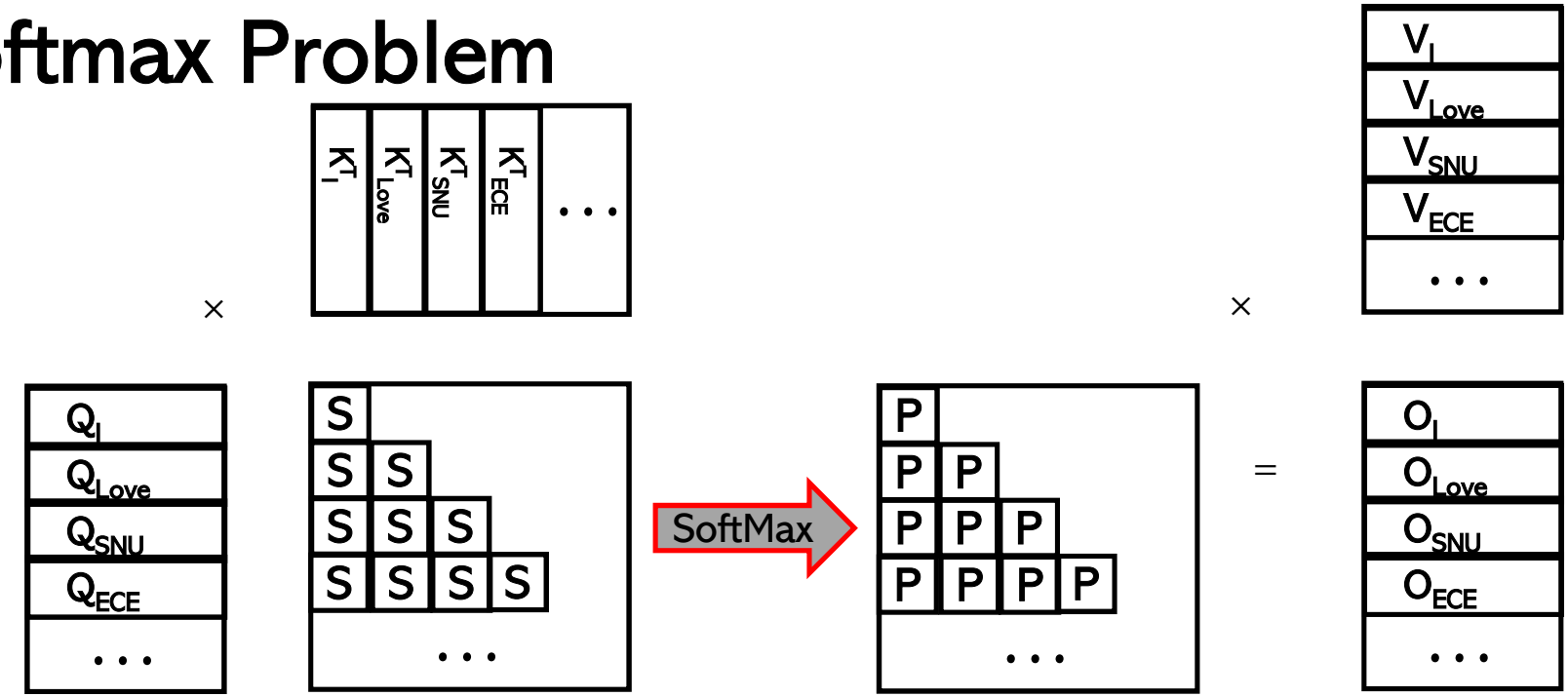
- Compute a block for S
- Use that block to compute a block of O
- Choose the tile to end up in the same O block
  - This will be important later

Block Size?



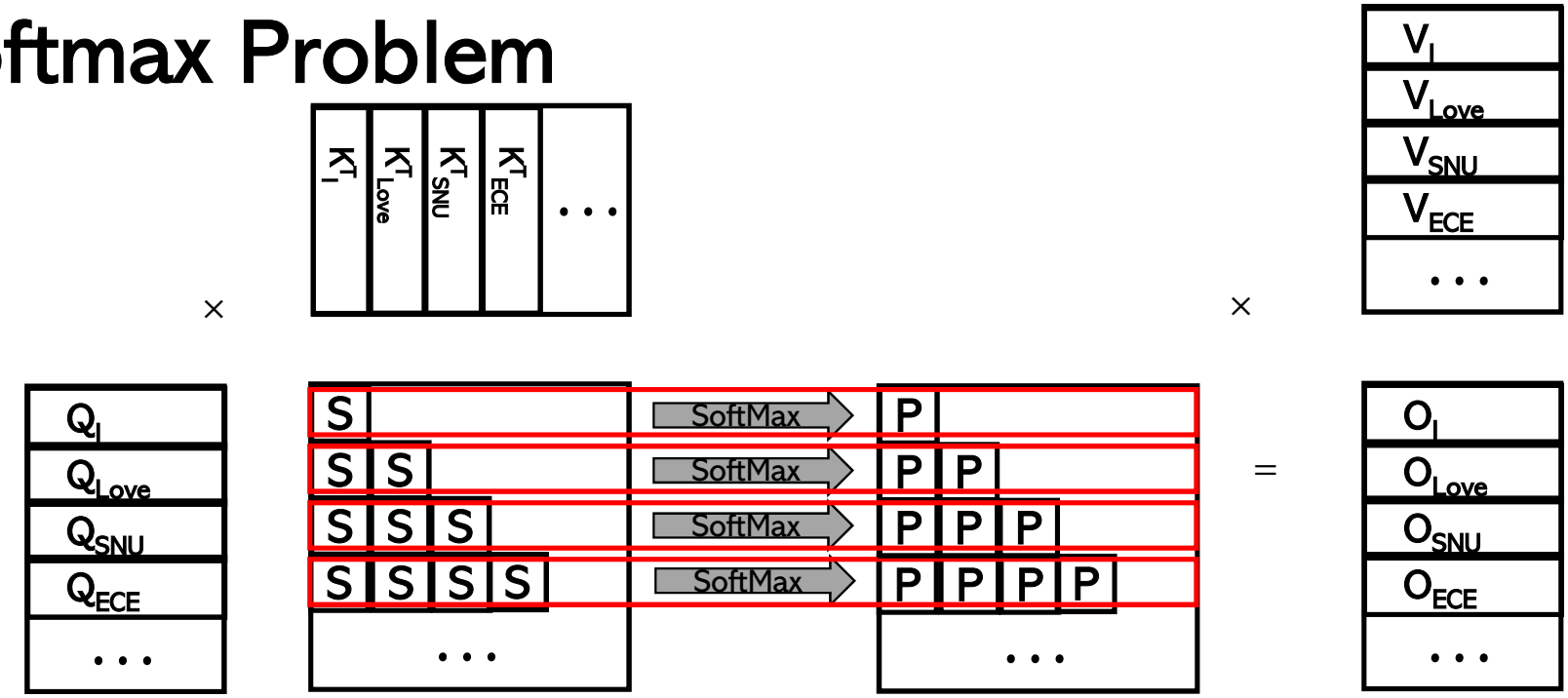
- Larger blocksize yields better performance
  - Aligns with what we know
  - It is bottlenecked at 256 for other factors
- Blocks larger than 512 don't fit in the shared memory

4. Softmax Problem



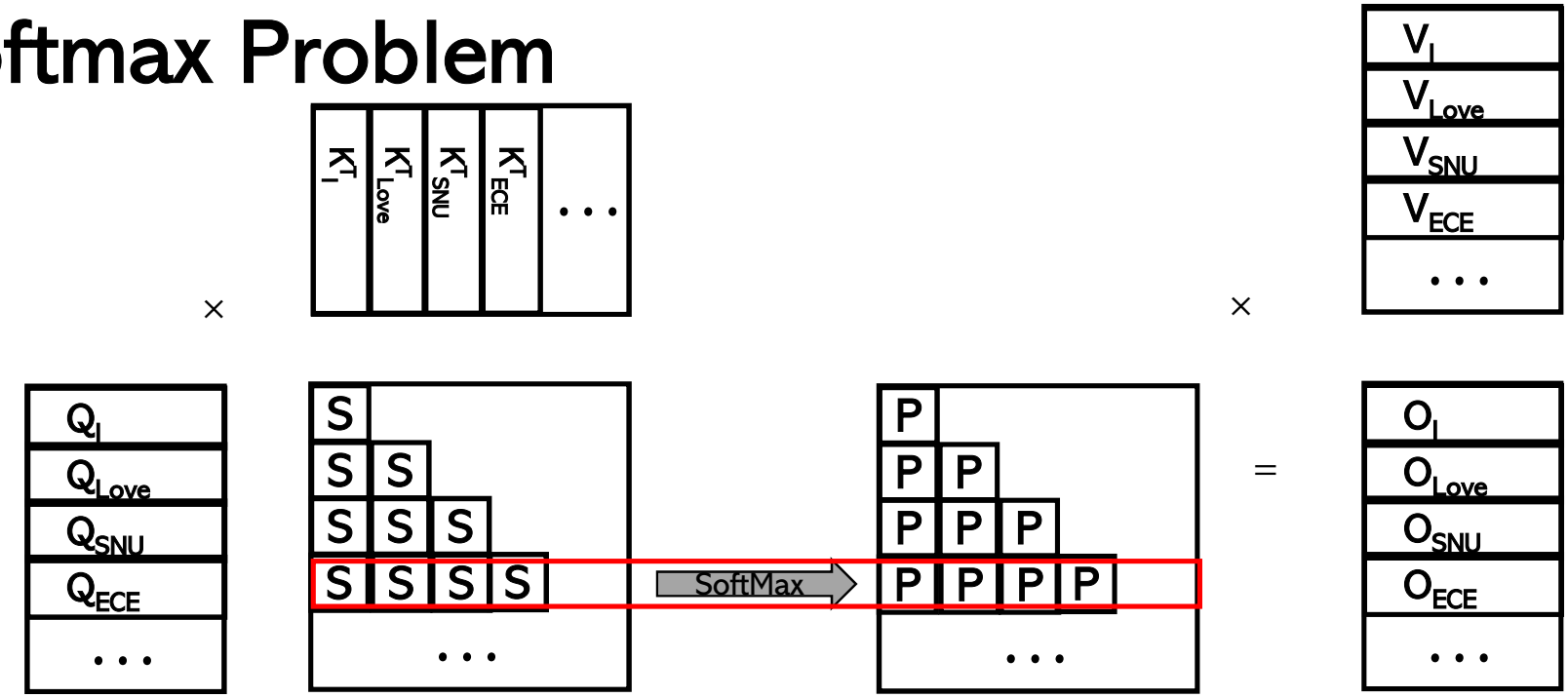
- Softmax is for converting S (Scores) into P (Probability)
- → Each O is a linear combination of V's with P being the weights

4. Softmax Problem



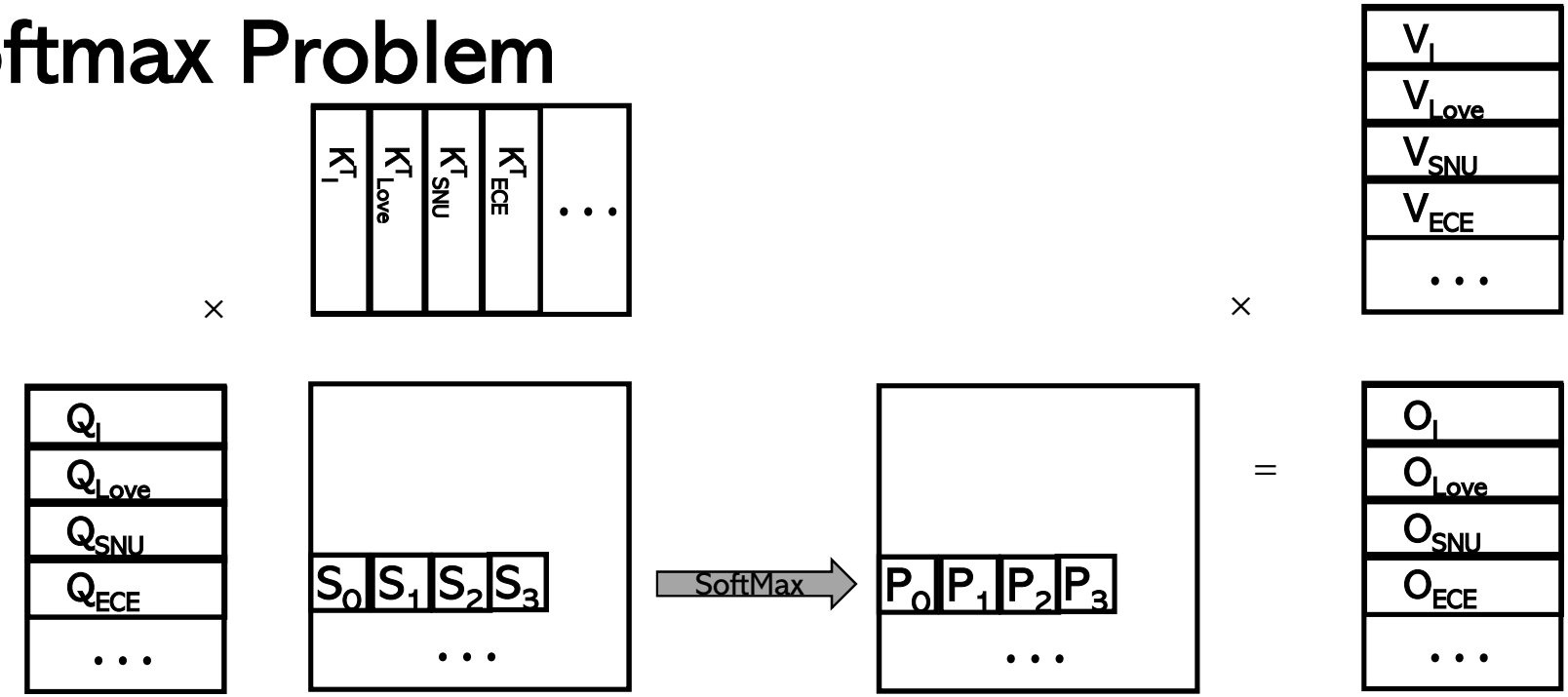
- Softmax is a row-wise operation
- $P_k = \frac{e^{S_k - m}}{\sum_i^N e^{S_i - m}}, \; m = \max_i(S_i)$

4. Softmax Problem



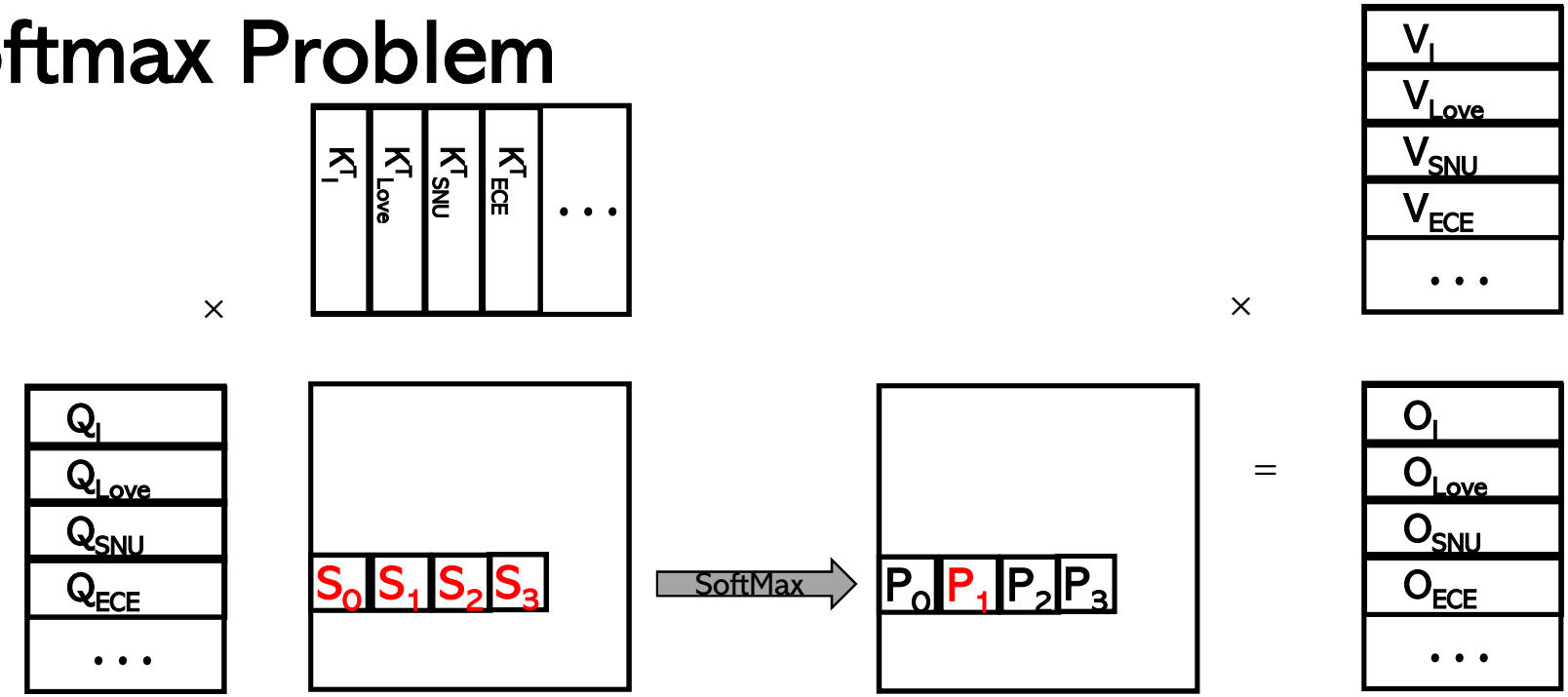
- Softmax is a row-wise operation
- $P_k = \frac{e^{S_k - m}}{\sum_i^N e^{S_i - m}}$, $\cancel{m = \max_i(S_i)}$ Ignore this for now, as it mathematically has no effect

4. Softmax Problem



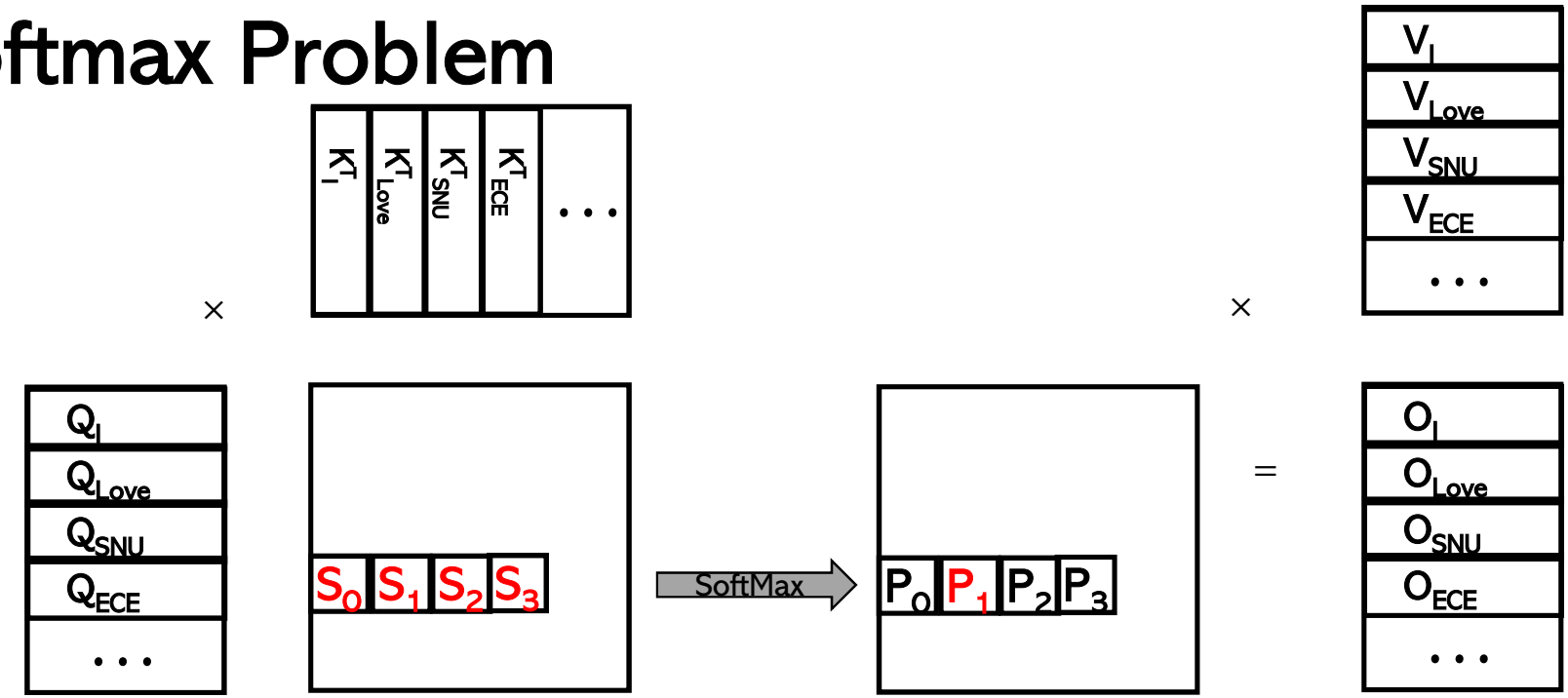
- Softmax is a row-wise operation
- $P_k = \frac{e^{S_k - m}}{\sum_i^N e^{S_i - m}}$, $\cancel{m = \max_i(S_i)}$ Ignore this for now, as it mathematically has no effect

4. Softmax Problem



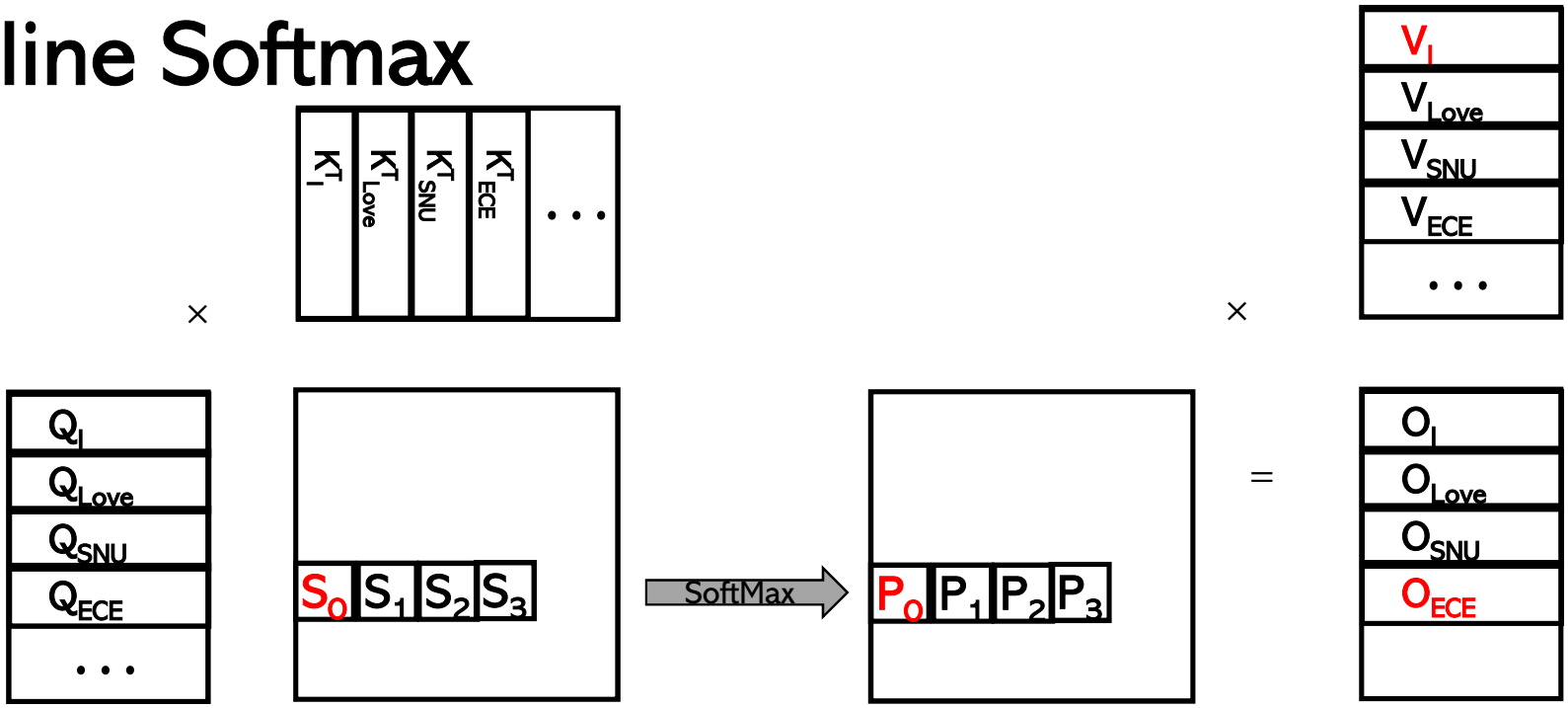
- Softmax is a row-wise operation
- $P_k = \frac{e^{S_k}}{\sum_i^N e^{S_i}}$
- $P_1 = \frac{e^{S_1}}{\sum_i^N e^{S_i}}$
- For the row sum, an entire row is needed
  → invalidates the kernel fusion technique

4. Softmax Problem



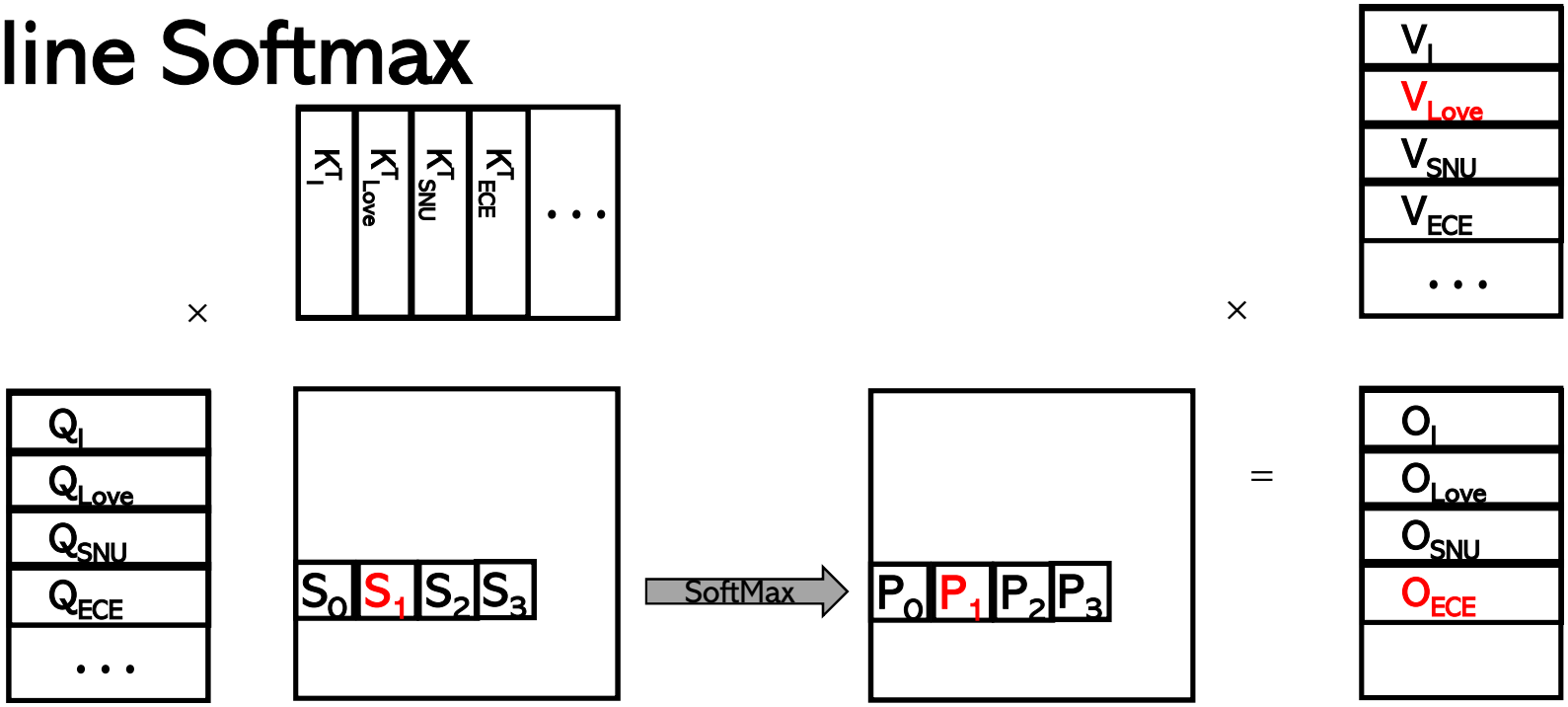
- Softmax is a row-wise operation
- $P_k = \frac{e^{S_k}}{\sum_i^N e^{S_i}}$
- Sol1: Compute S and row sum first, calculate P later (two-pass)
  - Problem: the row is too long (sequence length can go up to 1M)
  - → Does not fit in the shared memory
  - → S goes to the global memory, invalidating kernel fusion technique

4. Online Softmax



- $P_k = \frac{e^{S_k}}{\sum_i^N e^{S_i}}$
- Online Softmax
- $rowsum\ l_0 = e^{S_0}$
- $P_0 = \frac{e^{S_0}}{l_0} = \frac{e^{S_0}}{\sum_i^0 e^{S_i}}$ (wrong)
- $O_0 = P_0 \cdot V_0 = \frac{e^{S_0} \cdot V_0}{\sum_i^0 e^{S_i}}$ (partial)

4. Online Softmax



Only need to track latest l_i and O_i

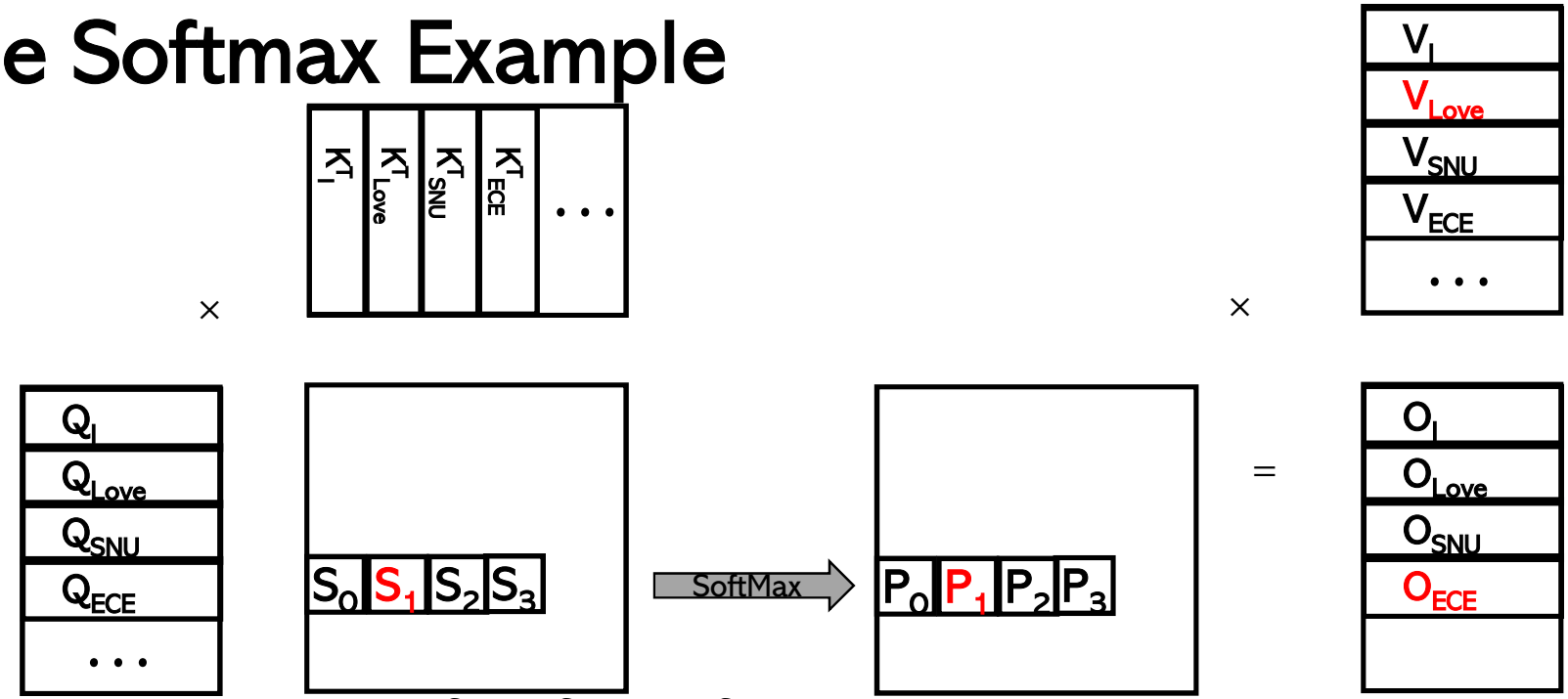
- $P_k = \frac{e^{S_k}}{\sum_i^N e^{S_i}}$
- Online Softmax
- $rowsum\ l_0 = e^{S_0}$
- $P_0 = \frac{e^{S_0}}{l_0} = \frac{e^{S_0}}{\sum_i^0 e^{S_i}}$
- $O_0 = P_0 \cdot V_0 = \frac{e^{S_0} \cdot V_0}{\sum_i^0 e^{S_i}}$

- $rowsum\ l_1 = l_0 + e^{S_1} = \sum_i^1 e^{S_i}$
- $P_1 = \frac{e^{S_1}}{l_1} = \frac{e^{S_1}}{\sum_i^1 e^{S_i}}$ (also wrong)
- $O_1 = O_0 \cdot \boxed{\frac{l_0}{l_1}} + P_1 \cdot V_1 = \frac{e^{S_0} \cdot V_0 + e^{S_1} \cdot V_1}{\sum_i^1 e^{S_i}}$

Denominator correction

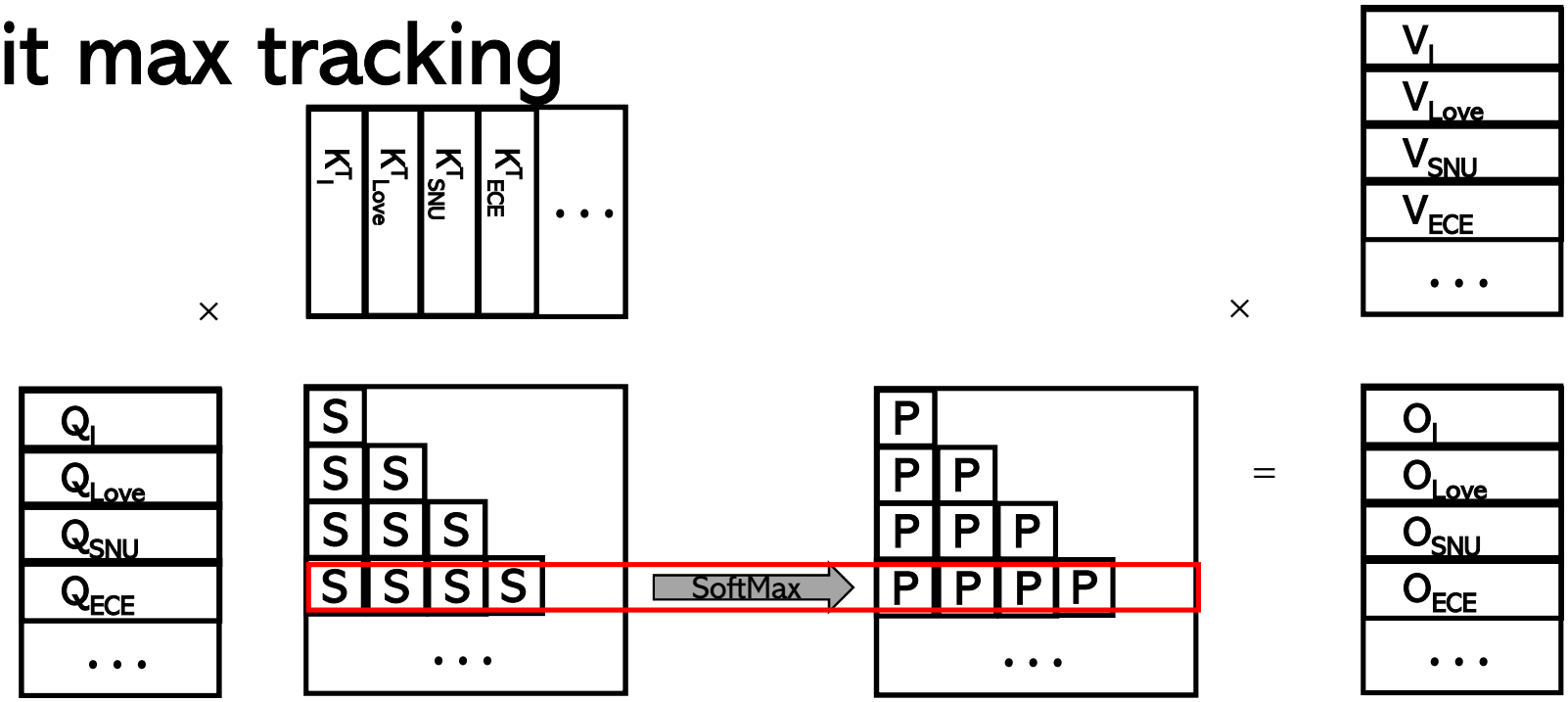
This becomes correct at the end (O_N)

Online Softmax Example



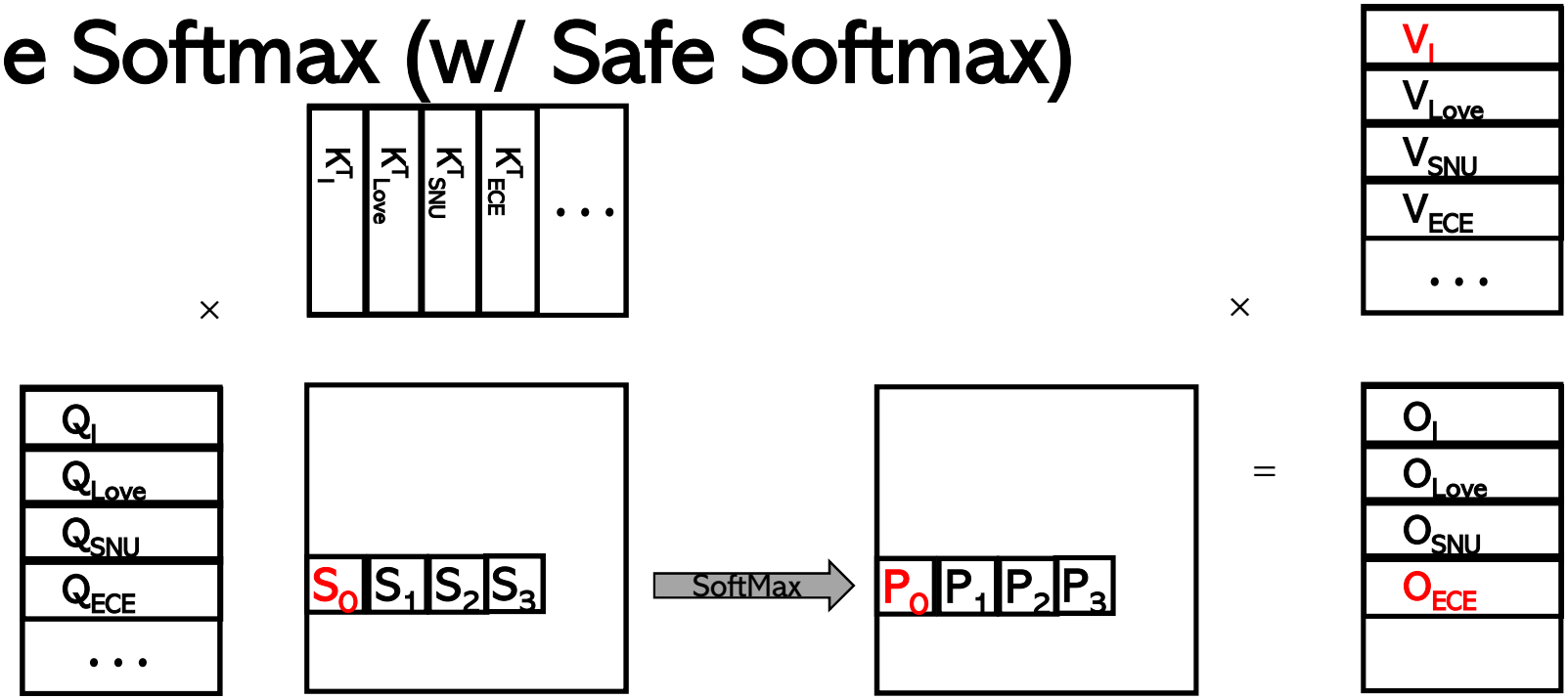
- $O_N = \frac{e^{S_0 \cdot V_0} + e^{S_1 \cdot V_1} + \cdots + e^{S_N \cdot V_N}}{l_N}$
- $O_0 = \frac{e^{S_0 \cdot V_0}}{l_0}$
- $O_1 = \frac{e^{S_0 \cdot V_0}}{\cancel{l_0}} \cdot \frac{\cancel{l_0}}{l_1} + \frac{e^{S_1 \cdot V_1}}{l_1} = \frac{e^{S_0 \cdot V_0} + e^{S_1 \cdot V_1}}{l_1}$
- $O_2 = \frac{e^{S_0 \cdot V_0} + e^{S_1 \cdot V_1}}{\cancel{l_1}} \cdot \frac{\cancel{l_1}}{l_2} + \frac{e^{S_2 \cdot V_2}}{l_2} = \frac{e^{S_0 \cdot V_0} + e^{S_1 \cdot V_1} + e^{S_2 \cdot V_2}}{l_2}$

Revisit max tracking



- Softmax is a row-wise operation
- $P_k = \frac{e^{S_k - m}}{\sum_i^N e^{S_i - m}}$, $m = \max_i(S_i)$ This is why really online softmax is needed
- m prevents e^{S_k} from overflowing fp32 or fp16
- Compute S → Compute m → compute P (three passes)
- (Safe Softmax)

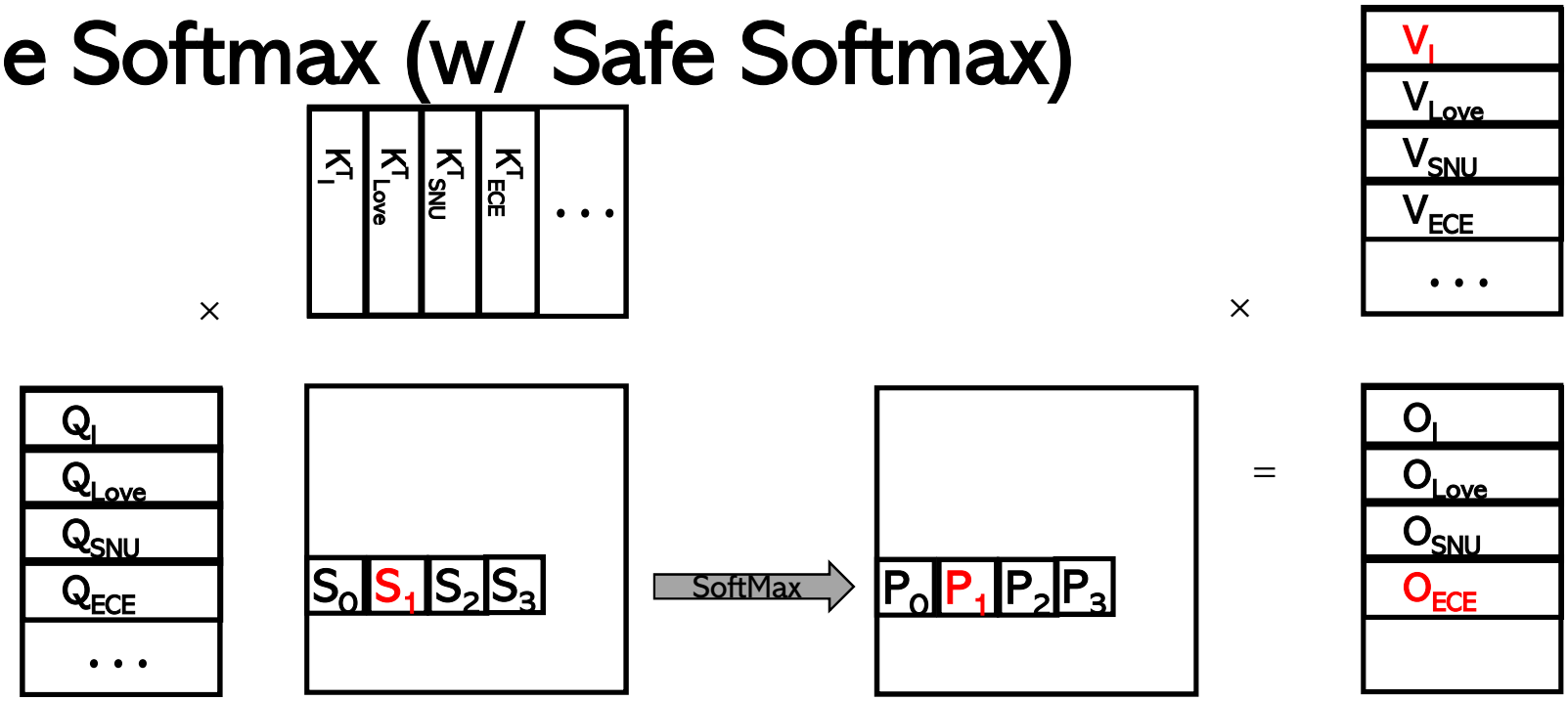
Online Softmax (w/ Safe Softmax)



- $P_k = \frac{e^{S_k - m}}{\sum_i^N e^{S_i - m}}$

- $m_0 = \max(S_0)$
- $rowsum\ l_0 = e^{S_0 - m_0}$
- $P_0 = \frac{e^{S_0 - m_0}}{l_0} = \frac{e^{S_0 - m_0}}{\sum_i^0 e^{S_i}}$ (wrong)
- $O_0 = P_0 \cdot V_0 = \frac{e^{S_0 - m_0} \cdot V_0}{\sum_i^0 e^{S_i - m_0}}$ (partial)

Online Softmax (w/ Safe Softmax)

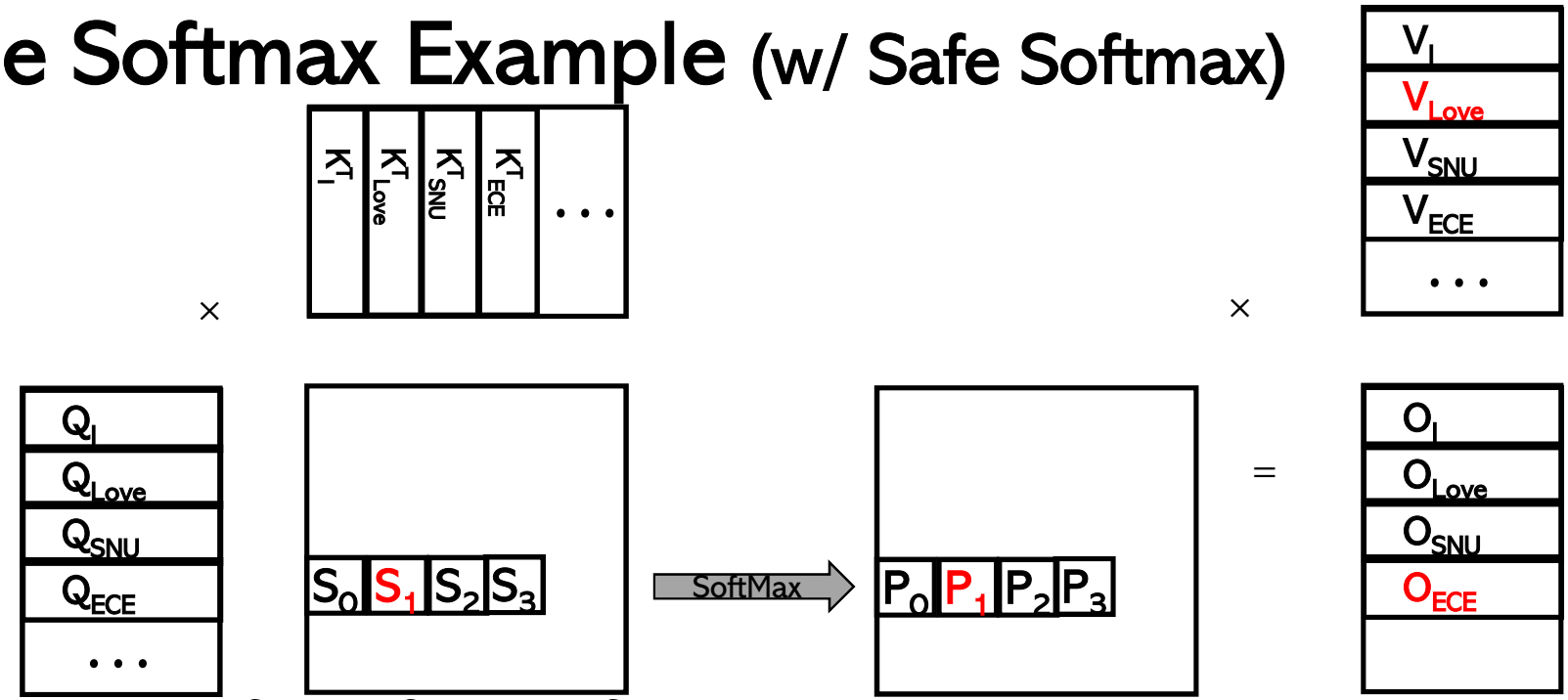


- $P_k = \frac{e^{S_k - m}}{\sum_i^N e^{S_i - m}}$

- $m_0 = \max(S_0)$
- $rowsum\ l_0 = e^{S_0 - m_0}$
- $P_0 = \frac{e^{S_0 - m_0}}{l_0} = \frac{e^{S_0 - m_0}}{\sum_i^0 e^{S_i - m_0}}$ (wrong)
- $O_0 = P_0 \cdot V_0 = \frac{e^{S_0 - m_0} \cdot V_0}{\sum_i^0 e^{S_i - m_0}}$ (partial)

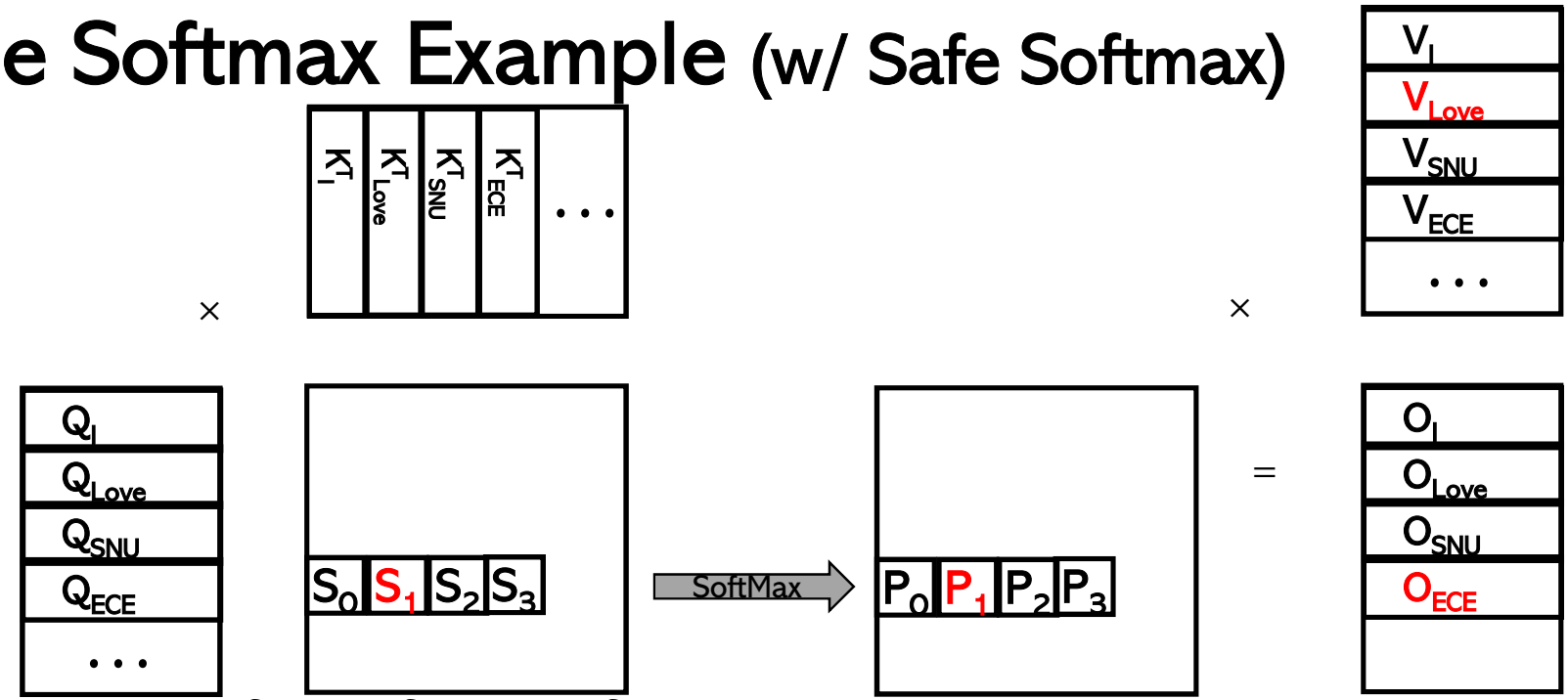
- $m_1 = \max(m_0, S_1)$
- $rowsum\ l_1 = l_0 \cdot \frac{m_0}{m_1} + e^{S_1 - m_1} = \sum_i^1 e^{S_i - m_1}$
- $P_1 = \frac{e^{S_1 - m_1}}{l_1} = \frac{e^{S_1 - m_1}}{\sum_i^1 e^{S_i - m_1}}$ (wrong)
- $O_1 = O_0 \cdot \frac{l_0}{l_1} \cdot e^{m_0 - m_1} + P_1 \cdot V_1 = \frac{e^{S_0 - m_1} \cdot V_0 + e^{S_1 - m_1} \cdot V_1}{\sum_i^1 e^{S_i - m_1}}$ (partial)

Online Softmax Example (w/ Safe Softmax)



- $O_N = \frac{e^{S_0-m}\cdot V_0+e^{S_1-m}\cdot V_1+\cdots+e^{S_N-m}\cdot V_N}{l_N}$
- $O_0 = \frac{e^{S_0-m_0}\cdot V_0}{l_0}$
- $O_1 = \frac{e^{S_0-m_0}\cdot V_0}{\cancel{l_0}}\cdot\frac{\cancel{l_0}}{l_1}\cdot e^{m_0-m_1} + \frac{e^{S_1-m_1}\cdot V_1}{l_1} = \frac{e^{S_0-m_1}\cdot V_0+e^{S_1-m_1}\cdot V_1}{l_1}$
- $O_2 = \frac{e^{S_0-m_1}\cdot V_0+e^{S_1-m_1}\cdot V_1}{\cancel{l_1}}\cdot\frac{\cancel{l_1}}{l_2}\cdot e^{m_1-m_2} + \frac{e^{S_2-m_2}\cdot V_2}{l_2} = \frac{e^{S_0-m_2}\cdot V_0+e^{S_1-m_2}\cdot V_1+e^{S_2-m_2}\cdot V_2}{l_2}$

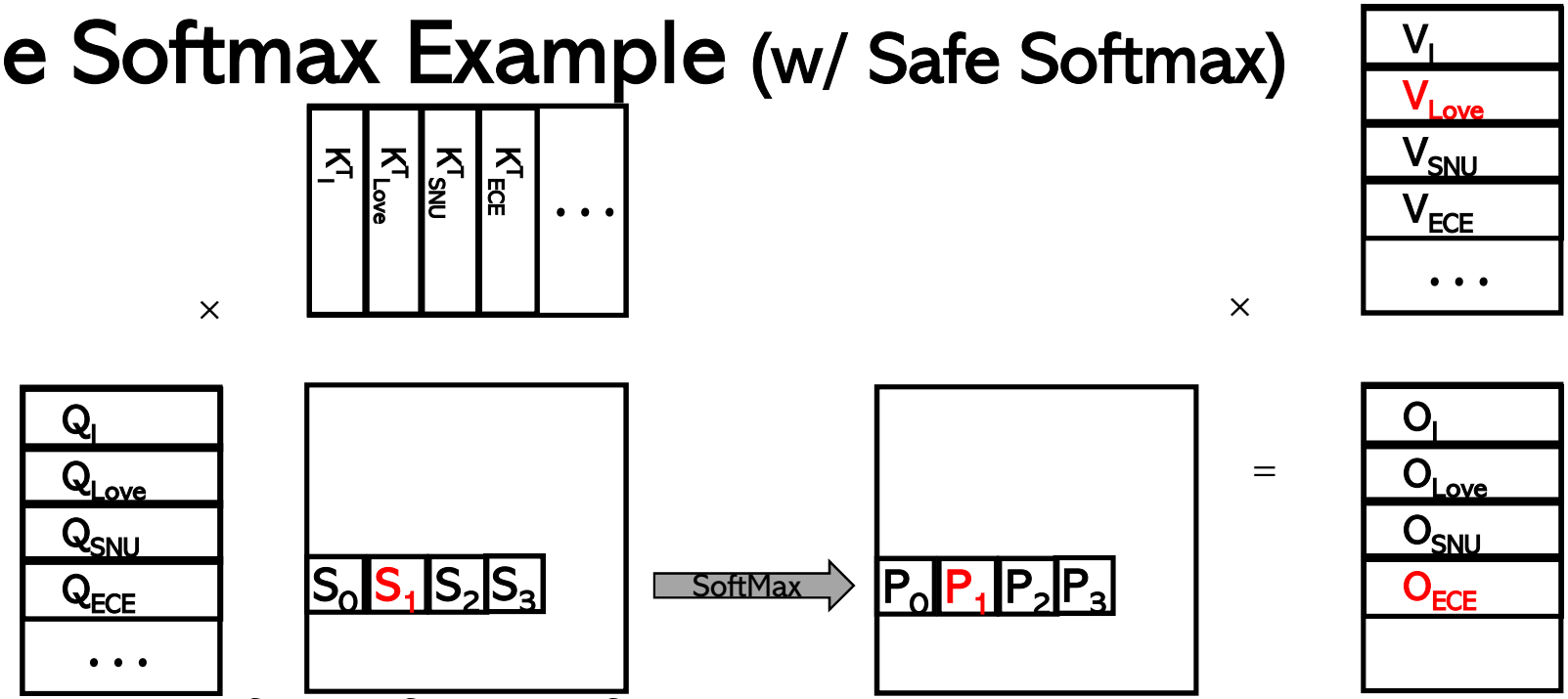
Online Softmax Example (w/ Safe Softmax)



- $O_N = \frac{e^{S_0-m}\cdot V_0+e^{S_1-m}\cdot V_1+\cdots+e^{S_N-m}\cdot V_N}{l_N}$
- $O_0 = \frac{e^{S_0-m_0}\cdot V_0}{l_0}$
- $O_1 = \frac{e^{S_0-m_0}\cdot V_0}{\cancel{l_0}}\cdot\frac{\cancel{l_0}}{l_1}\cdot e^{m_0-m_1} + \frac{e^{S_1-m_1}\cdot V_1}{l_1} = \frac{e^{S_0-m_1}\cdot V_0+e^{S_1-m_1}\cdot V_1}{l_1}$
- $O_2 = \frac{e^{S_0-m_1}\cdot V_0+e^{S_1-m_1}\cdot V_1}{\cancel{l_1}}\cdot\frac{\cancel{l_1}}{l_2}\cdot e^{m_1-m_2} + \frac{e^{S_2-m_2}\cdot V_2}{l_2} = \frac{e^{S_0-m_2}\cdot V_0+e^{S_1-m_2}\cdot V_1+e^{S_2-m_2}\cdot V_2}{l_2}$

We don't need to do this every time (division is expensive)
→ Just keep track of l and divide at the end

Online Softmax Example (w/ Safe Softmax)



- $O_N = \frac{e^{S_0-m}\cdot V_0+e^{S_1-m}\cdot V_1+\cdots+e^{S_N-m}\cdot V_N}{\cancel{l_N}}$

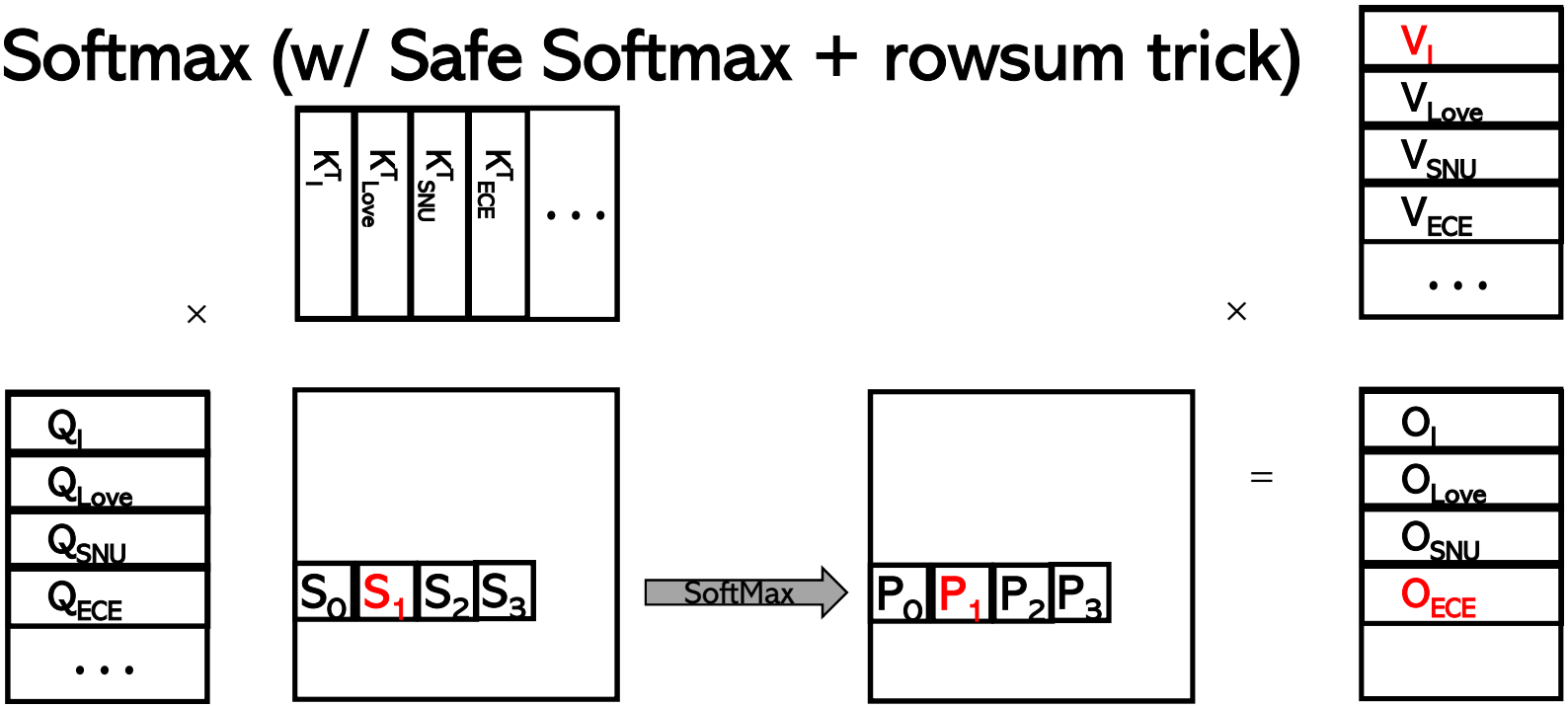
- $O_0 = \frac{e^{S_0-m_0}\cdot V_0}{\cancel{l_0}}$

We don't need to do this every time (division is expensive)
→ Just keep track of l and divide at the end

- $O_1 = \frac{e^{S_0-m_0}\cdot V_0}{\cancel{l_0}}\cdot\frac{\cancel{l_0}}{\cancel{l_1}}\cdot e^{m_0-m_1} + \frac{e^{S_1-m_1}\cdot V_1}{\cancel{l_1}} = \frac{e^{S_0-m_1}\cdot V_0+e^{S_1-m_1}\cdot V_1}{\cancel{l_1}}$

- $O_2 = \frac{e^{S_0-m_1}\cdot V_0+e^{S_1-m_1}\cdot V_1}{\cancel{l_1}}\cdot\frac{\cancel{l_1}}{\cancel{l_2}}\cdot e^{m_1-m_2} + \frac{e^{S_2-m_2}\cdot V_2}{\cancel{l_2}} = \frac{e^{S_0-m_2}\cdot V_0+e^{S_1-m_2}\cdot V_1+e^{S_2-m_2}\cdot V_2}{\cancel{l_2}}$

Online Softmax (w/ Safe Softmax + rowsum trick)



- $P_k = \frac{e^{S_k - m}}{\sum_i^N e^{S_i - m}}$

- $m_0 = \max(S_0)$
- $rowsum\ l_0 = e^{S_0 - m_0}$
- $P_0 = \frac{e^{S_0 - m_0}}{} = \frac{e^{S_0 - m_0}}{}$
- $O_0 = P_0 \cdot V_0 = \frac{e^{S_0 - m_0} \cdot V_0}{}$

- $m_1 = \max(m_0, S_1)$
- $rowsum\ l_1 = l_0 \cdot \frac{m_0}{m_1} + e^{S_1 - m_1} = \sum_i^1 e^{S_i - m_1}$
- $P_1 = \frac{e^{S_1 - m_1}}{} = \frac{e^{S_1 - m_1}}{}$ (without rowsum division)
- $O_1 = O_0 \cdot e^{m_0 - m_1} + P_1 \cdot V_1 = \frac{e^{S_0 - m_1} \cdot V_0 + e^{S_1 - m_1} \cdot V_1}{}$ (partial, without division)
- Later, $O_{Row} = O_N / l_N$

Summary

- Attention: a core operation for modern LLMs
- Attention gets more important for longer contexts
- The usual story
  - Parallelization
  - Tiling
  - Kernel fusion

  - With some mathematical trick (Online softmax) for making these work